

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO BÀI TẬP LỚN
HỆ THỐNG NHẬN DIỆN BIỂN SỐ XE
(LPR SYSTEM)

VIỆN TRÍ TUỆ NHÂN TẠO



Hà Nội, ngày 27 tháng 5 năm 2026

Mục lục

I	THÔNG TIN CHUNG VỀ DỰ ÁN	4
II	TỔNG QUAN DỰ ÁN	5
1	Lý do chọn đề tài	5
2	Mục tiêu dự án	5
3	Phạm vi dự án	5
4	Công nghệ sử dụng và luồng xử lý	6
a)	Công nghệ phía Backend (Python 3.10+)	6
b)	Công nghệ phía Frontend (React 18 + Vite 5)	8
c)	Công nghệ AI & Machine Learning	9
d)	Luồng xử lý hệ thống	10
III	PHÂN TÍCH YÊU CẦU	11
1	Yêu cầu chức năng	11
a)	Nhóm chức năng dành cho Khách (Guest)	11
b)	Nhóm chức năng dành cho Người dùng (User)	12
c)	Nhóm chức năng dành cho Quản trị viên (Admin)	12
2	Yêu cầu phi chức năng	13
IV	THIẾT KẾ HỆ THỐNG	14
1	Chi tiết các thành phần cấu trúc	14
a)	Tầng Trình diễn (Presentation Layer)	14
b)	Tầng Giao diện Lập trình Ứng dụng (API Layer)	15
c)	Tầng Nghiệp vụ (Service Layer)	16
d)	Tầng Truy cập Dữ liệu (Data Access Layer – DAL)	17
e)	Tầng Hạ tầng và Xử lý Bất đồng bộ (Infrastructure & Heavy Processing Microservices)	18
2	Hình ảnh tổng quan hệ thống	19
V	THIẾT KẾ CƠ SỞ DỮ LIỆU	19
1	Tổng quan cơ sở dữ liệu	20
2	Các nhóm thực thể chính	20
a)	Người dùng và xác thực	20
b)	Phương tiện và chủ sở hữu	21
c)	Phát hiện và ghi nhận biển số	22
d)	Quản lý vi phạm	23
e)	Danh sách đen	24
f)	Khiếu nại vi phạm	24
3	Các đặc điểm nổi bật trong thiết kế	25
4	Đánh giá thiết kế	25
VI	THIẾT KẾ PIPELINE AI	25
1	Tổng quan Pipeline AI	25
2	Kiến trúc mô hình AI	26
3	Pipeline xử lý ảnh	26
4	Pipeline xử lý video	27
5	Pipeline Webcam Realtime	27
6	Các kỹ thuật tối ưu hiệu năng	28

7	Đánh giá tổng kết Pipeline AI	29
VII	THIẾT KẾ API	30
1	Tổng quan hệ thống API	30
2	Authentication API	30
3	LPR Recognition API	30
4	Detection History API	31
5	Video Processing API	31
6	Traffic Violation API	31
7	Blacklist API	31
8	User Management API	32
9	WebSocket API	32
10	Đánh giá thiết kế API	32
VIII	THIẾT KẾ GIAO DIỆN	33
1	Giao diện đăng nhập	33
2	Giao diện đăng ký	34
3	Giao diện màn hình chính	34
4	Giao diện tìm kiếm lỗi vi phạm	35
5	Giao diện xem lịch sử tạo yêu cầu	36
6	Giao diện nhận diện biển số bằng ảnh	36
7	Giao diện nhận diện biển số bằng video	37
8	Giao diện lịch sử	37
9	Giao diện Blacklist	38
10	Giao diện webcam	38
11	Giao diện xử lý khiếu nại/nộp phạt	39
12	Giao diện quản lý danh sách người dùng	39
IX	MONITORING VÀ VẬN HÀNH	40
1	Mục tiêu monitoring	40
2	Kiến trúc monitoring	40
3	Các metrics chính	41
4	Dashboard Grafana	41
5	Cảnh báo Prometheus	41
6	Logging	42
7	Quy trình vận hành	42
X	TRIỂN KHAI DOCKER CI & CD	43
1	Mục tiêu triển khai	43
2	Docker hóa backend	43
3	Docker hóa frontend	43
4	Docker Compose production	44
5	Luồng CI/CD	44
6	Docker image registry	45
7	Quy trình CD trên server	45
8	Rollback	45
9	Bảo mật khi triển khai	45
XI	KIỂM THỬ VÀ ĐÁNH GIÁ	46
1	Mục tiêu kiểm thử	46

2	Môi trường kiểm thử	46
3	Các loại kiểm thử	46
4	Kiểm thử tự động	46
5	Kiểm thử chức năng	47
6	Đánh giá kết quả	47
7	Hạn chế	47
XII	BẢO MẬT VÀ RỦI RO	48
1	Các cơ chế bảo mật đã áp dụng	48
2	Rủi ro và biện pháp giảm thiểu	49
3	Khuyến nghị bảo mật khi triển khai production	49
XIII	PHÂN CÔNG NHÓM	50
1	Phân công nhóm	50
2	Kế hoạch thực hiện	50
3	Kịch bản demo	51

I THÔNG TIN CHUNG VỀ DỰ ÁN

Tên dự án: License Plate Recognition System

Môn học: Thực hành phát triển hệ thống AI

Năm học: 2025 – 2026

Giảng viên HD: Trịnh Ngọc Huỳnh

Nhóm thực hiện: 4conbo

THÀNH VIÊN THỰC HIỆN VÀ VAI TRÒ:

Họ và tên	Mã sinh viên	Vai trò
Phạm Văn Vương Thuận	24022461	Backend Developer, Testing
Trần Công Tuấn	24022485	Full-stack Developer, Database
Nguyễn Đắc Trung Hiếu	24022329	Frontend Developer, UI/UX
Trịnh Bình Minh	24022413	DevOps, AI Integration

II TỔNG QUAN DỰ ÁN

1 Lý do chọn đề tài

Hiện nay, số lượng phương tiện giao thông ngày càng tăng, kéo theo nhu cầu quản lý và giám sát giao thông hiệu quả hơn. Việc kiểm tra và xử lý phương tiện thủ công còn tốn nhiều thời gian, nhân lực và dễ xảy ra sai sót.

Nhờ sự phát triển của trí tuệ nhân tạo (AI) và thị giác máy tính (Computer Vision), các hệ thống nhận diện biển số xe tự động đã được ứng dụng rộng rãi trong giám sát giao thông, bãi đỗ xe thông minh và kiểm soát an ninh. Công nghệ này giúp nhận diện phương tiện nhanh chóng, chính xác và tự động hóa quá trình quản lý.

Vì vậy, nhóm lựa chọn đề tài xây dựng hệ thống nhận diện biển số xe nhằm nghiên cứu và ứng dụng các kỹ thuật xử lý ảnh, học sâu và nhận dạng ký tự vào bài toán thực tế.

2 Mục tiêu dự án

Mục tiêu chính của dự án là xây dựng hệ thống nhận diện biển số xe thông minh, hỗ trợ quản lý vi phạm giao thông và điểm phạt theo quy định 12 điểm của Luật Giao thông Việt Nam.

Các mục tiêu cụ thể bao gồm:

- Nhận diện biển số xe từ ảnh tĩnh, video và webcam theo thời gian thực.
- Quản lý danh sách đen (blacklist) và lưu trữ lịch sử phát hiện phương tiện.
- Xây dựng hệ thống quản lý vi phạm, hỗ trợ khiếu nại và phân quyền người dùng.
- Thiết kế giao diện thân thiện, hỗ trợ chế độ sáng/tối (light mode/dark mode).

3 Phạm vi dự án

Dự án tập trung xây dựng hệ thống nhận diện biển số xe áp dụng cho môi trường giao thông tại Việt Nam. Hệ thống hỗ trợ xử lý dữ liệu đầu vào từ ảnh, video và webcam để phát hiện và nhận diện biển số xe.

Đối tượng sử dụng chính bao gồm quản trị viên, cán bộ quản lý giao thông và người dùng có nhu cầu tra cứu thông tin vi phạm. Hệ thống cung cấp các chức năng quản lý người dùng, lưu trữ lịch sử nhận diện, quản lý vi phạm và hỗ trợ khiếu nại.

Trong phạm vi đề tài, hệ thống chủ yếu phục vụ mục đích nghiên cứu và thử nghiệm, chưa triển khai trên quy mô lớn hoặc tích hợp với cơ sở dữ liệu giao thông quốc gia.

4 Công nghệ sử dụng và luồng xử lý

Hệ thống nhận diện biển số xe (License Plate Recognition System) được xây dựng theo kiến trúc Microservices, tách biệt giữa Frontend và Backend nhằm tăng khả năng mở rộng, bảo trì và tối ưu hiệu năng xử lý. Hệ thống tích hợp các mô hình Deep Learning định dạng .pt/.onnx kết hợp cơ chế xử lý bất đồng bộ để đáp ứng yêu cầu nhận diện ảnh và video theo thời gian thực.

a) Công nghệ phía Backend (Python 3.10+)

Framework & Server

- **FastAPI:** Framework web chính, hỗ trợ xử lý bất đồng bộ (*async*) và tự động sinh tài liệu API theo chuẩn OpenAPI Docs.
- **Uvicorn:** ASGI Server dùng để triển khai và vận hành ứng dụng FastAPI, đồng thời hỗ trợ giao tiếp WebSocket.
- **Pydantic v2:** Thư viện kiểm định dữ liệu (*Data Validation*) và định nghĩa các mô hình Request/Response Schema.

Database & ORM

- **SQLAlchemy 2.0:** Thư viện ORM chính dùng để tương tác và quản lý cơ sở dữ liệu thông qua mô hình đối tượng trong Python.
- **PostgreSQL:** Hệ quản trị cơ sở dữ liệu quan hệ chính của hệ thống, dùng để lưu trữ dữ liệu người dùng, lịch sử xử lý và thông tin nhận diện.
- **Supabase:** Nền tảng Backend-as-a-Service sử dụng PostgreSQL, hỗ trợ lưu trữ cơ sở dữ liệu trên nền tảng đám mây và cung cấp khả năng quản lý dữ liệu trực tuyến.
- **psycopg2-binary:** Driver kết nối giữa ứng dụng Python và cơ sở dữ liệu PostgreSQL/Supabase.
- **aiosqlite:** Driver SQLite hỗ trợ lập trình bất đồng bộ, chủ yếu phục vụ cho quá trình kiểm thử hệ thống.

Authentication & Security

- **python-jose:** Thư viện xử lý mã hóa và giải mã JWT (*JSON Web Token*) phục vụ xác thực người dùng.
- **passlib + bcrypt:** Công cụ băm (*hash*) và xác thực mật khẩu người dùng.
- **python-dotenv:** Hỗ trợ đọc và quản lý các biến môi trường từ tệp cấu hình .env.

Task Queue & Real-time

- **Celery:** Hệ thống hàng đợi tác vụ (*Task Queue*) dùng để xử lý các tác vụ nặng bất đồng bộ như xử lý video hoặc AI inference.
- **Redis:** Đóng vai trò Message Broker, lưu trữ kết quả (*Result Backend*) và hỗ trợ cơ chế Pub/Sub.
- **WebSocket (FastAPI):** Hỗ trợ cập nhật tiến trình xử lý dữ liệu theo thời gian thực (*Real-time*) tới phía Client.

Object Storage

- **MinIO**: Hệ thống lưu trữ đối tượng dùng để quản lý hình ảnh, video và dữ liệu đa phương tiện, tương thích chuẩn Amazon S3.
- **minio-py**: Python SDK hỗ trợ giao tiếp và thao tác trực tiếp với MinIO.

Testing

- **pytest**: Framework chính dùng để xây dựng và thực thi Unit Test và Integration Test.
- **pytest-cov**: Công cụ đo lường độ bao phủ mã nguồn (*Code Coverage*).
- **httpx**: HTTP Client hỗ trợ bất đồng bộ, dùng để kiểm thử các API Async.

Các công cụ hỗ trợ khác

- **python-multipart**: Hỗ trợ xử lý dữ liệu Form-data và tải tệp tin lên hệ thống.
- **pandas**: Thư viện xử lý dữ liệu và xuất lịch sử phát hiện ra định dạng CSV.
- **matplotlib / seaborn**: Hỗ trợ trực quan hóa dữ liệu và vẽ biểu đồ thống kê.
- **tqdm**: Hiển thị thanh tiến trình (*Progress Bar*) trên giao diện dòng lệnh.
- **PyYAML**: Hỗ trợ đọc và xử lý các tệp cấu hình định dạng YAML của YOLOv5.

Bảng 1: Bảng tổng hợp các công nghệ nổi bật trong Backend

Công nghệ	Phiên bản	Mục đích sử dụng
FastAPI	0.115+	Framework web chính hỗ trợ xây dựng RESTful API bất đồng bộ và tự động sinh tài liệu OpenAPI.
Uvicorn	0.30+	ASGI Server dùng để triển khai và vận hành ứng dụng FastAPI.
SQLAlchemy	2.0+	ORM dùng để thao tác và quản lý cơ sở dữ liệu bằng Python.
Celery	5.4+	Hệ thống hàng đợi tác vụ dùng để xử lý các công việc nền bất đồng bộ.
Redis	7.0+	Message Broker, Result Backend và hỗ trợ Pub/Sub cho hệ thống.
WebSocket (FastAPI)	Built-in	Gửi tiến trình xử lý dữ liệu thời gian thực tới Client.
MinIO	RELEASE 2025+	Hệ thống Object Storage lưu trữ ảnh và video theo chuẩn S3.
pytest	8.0+	Framework viết và chạy Unit Test, Integration Test.
httpx	0.27+	Async HTTP Client dùng để kiểm thử API.

b) Công nghệ phía Frontend (React 18 + Vite 5)

Khung phát triển cốt lõi (Core Framework)

Phần Frontend của hệ thống được xây dựng dựa trên thư viện React kết hợp cùng các công cụ hỗ trợ tối ưu hiệu năng và quản lý điều hướng ứng dụng đơn trang (SPA).

Bảng 2: Các công nghệ cốt lõi phía Frontend

STT	Công nghệ	Phiên bản	Mục đích sử dụng
1	React	18.3.1	Thư viện chính dùng để xây dựng giao diện người dùng (UI Components) và quản lý vòng đời ứng dụng thông qua Hooks.
2	Vite	5.4.0	Công cụ Build Tool hiện đại hỗ trợ HMR (Hot Module Replacement) nhanh và tối ưu hóa mã nguồn khi triển khai.
3	React Router DOM	6.26.2	Hệ thống điều hướng và quản lý định tuyến phía Client cho ứng dụng SPA.

Thiết kế giao diện (Styling & UI)

Giao diện hệ thống được thiết kế theo hướng hiện đại, thân thiện với người dùng và hỗ trợ khả năng hiển thị thích ứng trên nhiều thiết bị khác nhau.

Bảng 3: Các công nghệ thiết kế giao diện

STT	Công nghệ	Phiên bản	Mục đích sử dụng
1	Tailwind CSS	4.3.0	Framework CSS theo hướng utility-first giúp xây dựng giao diện nhanh chóng và tối ưu dung lượng CSS.
2	DaisyUI	5.5.19	Thư viện UI Component xây dựng trên nền Tailwind CSS, cung cấp các thành phần giao diện đồng bộ và hiện đại.
3	Lucide React	1.14.0	Thư viện icon SVG mã nguồn mở hỗ trợ hiển thị sắc nét trên nhiều độ phân giải màn hình.

Quản lý trạng thái và giao tiếp API (State Management & HTTP)

Hệ thống ưu tiên sử dụng các công cụ tích hợp sẵn nhằm tối ưu hiệu năng và giảm sự phụ thuộc vào các thư viện bên ngoài.

Bảng 4: Các công nghệ quản lý trạng thái và giao tiếp API

STT	Công nghệ	Phiên bản	Mục đích sử dụng
1	React Context	Tích hợp sẵn	Quản lý trạng thái toàn cục của ứng dụng, cụ thể là chuyển đổi giao diện sáng/tối (Theme Context).
2	Fetch API	Tích hợp sẵn	Gửi và xử lý các yêu cầu HTTP tới Backend API mà không cần sử dụng thư viện Axios.
3	localStorage	Trình duyệt	Lưu trữ cục bộ thông tin phiên đăng nhập như JWT Token, Role và Username.

c) Công nghệ AI & Machine Learning

Công nghệ và thư viện hỗ trợ

Hệ thống sử dụng các thư viện và framework phổ biến trong lĩnh vực Thị giác máy tính (*Computer Vision*) nhằm phục vụ quá trình huấn luyện và triển khai mô hình AI.

Bảng 5: Các công nghệ và thư viện AI sử dụng trong hệ thống

STT	Công nghệ / Thư viện	Phiên bản	Mục đích sử dụng
1	YOLOv5 (Custom)	Tùy chỉnh	Mô hình phát hiện vùng chứa biển số xe trong ảnh và video.
3	PyTorch	2.x	Framework dùng để xây dựng và huấn luyện mô hình học sâu.
4	TorchVision	0.x	Hỗ trợ tiền xử lý ảnh và biến đổi dữ liệu cho mô hình AI.
5	OpenCV	4.x	Xử lý ảnh, đọc video và thao tác trên từng khung hình.
6	NumPy	Mới nhất	Xử lý ma trận ảnh và các phép toán đại số tuyến tính.
7	Pillow (PIL)	Mới nhất	Hỗ trợ đọc, ghi và chuyển đổi định dạng ảnh.

Danh sách mô hình huấn luyện sẵn

Hệ thống triển khai kiến trúc hai giai đoạn (*Two-stage Detector*) gồm mô hình phát hiện biển số và mô hình nhận diện ký tự.

Bảng 6: Danh sách mô hình AI được sử dụng

STT	Tên mô hình	Định dạng	Dung lượng	Chức năng
1	LP_detector_nano_61	.pt	~3.67 MB	Phát hiện và trả về tọa độ vùng chứa biển số xe.
2	LP_ocr_nano_62	.pt	~3.9 MB	Nhận diện ký tự từ vùng ảnh biển số đã cắt.

d) Luồng xử lý hệ thống

Hệ thống hoạt động theo quy trình xử lý tuần tự từ lúc tiếp nhận dữ liệu đến khi trả kết quả nhận diện cho người dùng:

1. **Tiếp nhận dữ liệu:** Người dùng tải ảnh, video hoặc sử dụng webcam để cung cấp dữ liệu đầu vào cho hệ thống.
2. **Phát hiện biển số xe:** Backend gửi dữ liệu hình ảnh đến mô hình YOLOv5 nhằm xác định chính xác vị trí biển số trong ảnh hoặc video.
3. **Nhận diện ký tự:** Sau khi phát hiện biển số, hệ thống tự động cắt vùng ảnh chứa biển số (crop) và đưa vào mô hình OCR để nhận diện ký tự.
4. **Lưu trữ dữ liệu:** Kết quả nhận diện được lưu trong PostgreSQL/Supabase, đồng thời ảnh gốc và ảnh cắt được lưu trên MinIO.
5. **Đối chiếu và xử lý thông tin:** Hệ thống kiểm tra blacklist, lịch sử vi phạm và các thông tin liên quan trước khi trả kết quả về giao diện người dùng.
6. **Xử lý bất đồng bộ:** Đối với video hoặc tác vụ lớn, Celery kết hợp Redis được sử dụng để xử lý nền nhằm giảm tải và tăng hiệu năng hệ thống.

III PHÂN TÍCH YÊU CẦU

Phân tích yêu cầu là giai đoạn quan trọng trong quá trình phát triển hệ thống, nhằm xác định rõ các chức năng cần xây dựng, đối tượng sử dụng và các yêu cầu kỹ thuật cần đáp ứng. Đối với hệ thống nhận diện biển số xe thông minh, việc phân tích yêu cầu giúp đảm bảo hệ thống hoạt động ổn định, chính xác và phù hợp với nhu cầu quản lý giao thông thực tế.

Trong hệ thống này, nhóm sẽ trình bày các **yêu cầu chức năng**, **yêu cầu phi chức năng**, **mô hình hoạt động** cũng như các thành phần chính của hệ thống nhằm làm cơ sở cho quá trình thiết kế và triển khai ở các chương tiếp theo.

1 Yêu cầu chức năng

Yêu cầu chức năng mô tả các hành vi và dịch vụ mà hệ thống phải cung cấp cho từng nhóm người dùng. Hệ thống được chia thành ba nhóm chức năng chính gồm: Khách (*Guest*), Người dùng (*User*) và Quản trị viên (*Admin*).

a) Nhóm chức năng dành cho Khách (*Guest*)

Bảng 7: Các chức năng dành cho khách

STT	Mã UC	Tên chức năng	Mô tả chức năng
1	UC-01	Đăng ký tài khoản	Cho phép người dùng tạo tài khoản mới bằng Username và Password.
2	UC-02	Đăng nhập hệ thống	Xác thực thông tin đăng nhập và cấp JWT Token khi đăng nhập thành công.

b) Nhóm chức năng dành cho Người dùng (User)

Bảng 8: Các chức năng dành cho người dùng

STT	Mã UC	Tên chức năng	Mô tả chức năng
1	UC-03	Xem Dashboard	Hiển thị thống kê tổng quan và biểu đồ dữ liệu hệ thống.
2	UC-04	Nhận diện từ ảnh	Tải ảnh lên và thực hiện nhận diện biển số xe.
3	UC-05	Nhận diện từ video	Xử lý video bất đồng bộ và trả kết quả theo từng frame.
4	UC-06	Nhận diện thời gian thực	Thu thập dữ liệu Webcam và trả kết quả real-time.
5	UC-07	Xem lịch sử phát hiện	Tra cứu lịch sử nhận diện và tìm kiếm theo bộ lọc.
6	UC-08	Xem danh sách đen	Hiển thị danh sách biển số bị cảnh báo (Read-only).
7	UC-09	Tra cứu vi phạm	Tra cứu thông tin vi phạm và điểm phạt của phương tiện.
8	UC-10	Gửi khiếu nại	Tạo và gửi đơn khiếu nại đối với biên bản vi phạm.
9	UC-11	Xem lịch sử khiếu nại	Theo dõi trạng thái xử lý các đơn khiếu nại đã gửi.

c) Nhóm chức năng dành cho Quản trị viên (Admin)

Bảng 9: Các chức năng dành cho quản trị viên

STT	Mã UC	Tên chức năng	Mô tả chức năng
1	UC-12	Quản lý vi phạm	Thêm, sửa, xóa các lỗi vi phạm và mức xử phạt tương ứng.
2	UC-13	Phê duyệt khiếu nại	Xét duyệt hoặc từ chối các đơn khiếu nại từ người dùng.
3	UC-14	Quản lý danh sách đen	Cập nhật dữ liệu biển số thuộc danh sách đen của hệ thống.
4	UC-15	Quản lý người dùng	Quản lý tài khoản và phân quyền User/Admin.
5	UC-16	Xuất dữ liệu báo cáo	Xuất dữ liệu lịch sử phát hiện và vi phạm ra file CSV.

2 Yêu cầu phi chức năng

Yêu cầu phi chức năng mô tả các tiêu chuẩn kỹ thuật và chất lượng nhằm đảm bảo hệ thống hoạt động ổn định, an toàn và có khả năng mở rộng.

Bảng 10: Các yêu cầu phi chức năng của hệ thống

STT	Mã YC	Tiêu chí	Đặc tả yêu cầu kỹ thuật
1	NFR-01	Hiệu năng	• Thời gian xử lý ảnh nhỏ hơn 2 giây. • Video được xử lý bất đồng bộ nhằm tránh nghẽn hệ thống API.
2	NFR-02	Bảo mật	• Mật khẩu được mã hóa bằng Bcrypt. • JWT Token có thời hạn sử dụng. • Kiểm soát truy cập theo vai trò (RBAC).
3	NFR-03	Khả dụng	• Hệ thống đảm bảo uptime cao. • Celery hỗ trợ cơ chế Retry khi gặp lỗi.
4	NFR-04	Giao diện	• Responsive UI cho Desktop và Mobile. • Hỗ trợ Light/Dark Mode.
5	NFR-05	Khả năng mở rộng	• Hỗ trợ kiến trúc Microservices. • Có thể mở rộng số lượng Celery Workers khi tải hệ thống tăng cao.

IV THIẾT KẾ HỆ THỐNG

Để đáp ứng các yêu cầu về khả năng mở rộng (scalability), tính sẵn sàng cao (high availability) và tối ưu hóa hiệu năng xử lý luồng dữ liệu video/hình ảnh thời gian thực, hệ thống LPR được thiết kế dựa trên sự kết hợp giữa mô hình Kiến trúc đa tầng (3-Layer Architecture) truyền thống ở cấp độ monolithic-core và Mô hình vi dịch vụ kết hợp xử lý bất đồng bộ (Microservices & Asynchronous Pattern) dành riêng cho các tác vụ tính toán nặng.

1 Chi tiết các thành phần cấu trúc

Kiến trúc hệ thống được phân rã thành các tầng chức năng độc lập nhằm đảm bảo nguyên lý *Single Responsibility Principle* và giảm thiểu sự phụ thuộc lẫn nhau (*Loose Coupling*).

a) Tầng Trình diễn (Presentation Layer)

- **Công nghệ sử dụng:** ReactJS, Vite
- **Vai trò:** Đóng vai trò là *Single Page Application (SPA)*, chịu trách nhiệm xây dựng giao diện người dùng (*UI/UX*) trực quan, hiện đại và tối ưu tốc độ phản hồi.
- **Chức năng chính:**
 - Tiếp nhận tương tác từ các Actor: *Guest, User, Admin*.
 - Gửi các yêu cầu HTTP Request đến API Layer dưới dạng:
 - * JSON
 - * *multipart/form-data* (khi tải tệp lên hệ thống)
 - Hiển thị kết quả xử lý và phân tích dữ liệu theo thời gian thực cho người dùng.
 - Quản lý điều hướng trang bằng cơ chế SPA giúp giảm thời gian tải lại trang.
- **Ưu điểm:**
 - Tăng trải nghiệm người dùng nhờ tốc độ phản hồi nhanh.
 - Tách biệt rõ ràng giữa giao diện và xử lý nghiệp vụ.
 - Dễ mở rộng và bảo trì hệ thống.

b) Tầng Giao diện Lập trình Ứng dụng (API Layer)

- **Công nghệ sử dụng:** Uvicorn ASGI Server, FastAPI Framework
- **Vai trò:** Đóng vai trò là lớp trung gian (*Middleware Layer*) tiếp nhận toàn bộ yêu cầu từ tầng Trình diễn và điều phối dữ liệu đến các dịch vụ xử lý nghiệp vụ phía sau.
- **Chức năng chính:**
 - **FastAPI Routers:**
 - * Thực hiện phân tuyến (*Routing*) các luồng xử lý đến từng API Endpoint tương ứng.
 - * Tổ chức API theo từng module chức năng: *Authentication*, *License Plate Recognition (LPR)*, *Traffic Monitoring*, *Blacklist Management*, *Video Processing*.
 - * Chuẩn hóa cấu trúc RESTful API giúp dễ dàng mở rộng và bảo trì hệ thống.
 - **Middleware Authentication:**
 - * Tích hợp cơ chế xác thực JWT (*JSON Web Token*) nhằm kiểm tra tính hợp lệ của người dùng.
 - * Áp dụng mô hình phân quyền RBAC (*Role-based Access Control*) để kiểm soát quyền truy cập giữa các nhóm: *Guest*, *User*, *Admin*.
 - * Bảo vệ các API quan trọng khỏi truy cập trái phép.
 - **Request Validation:**
 - * Sử dụng Pydantic Schema để kiểm tra dữ liệu đầu vào.
 - * Hạn chế lỗi dữ liệu không hợp lệ trước khi chuyển sang tầng xử lý nghiệp vụ.
 - **Response Handling:**
 - * Chuẩn hóa dữ liệu phản hồi dưới định dạng JSON.
 - * Trả về mã trạng thái HTTP phù hợp cho từng trường hợp xử lý.
 - **Asynchronous Processing:**
 - * Hỗ trợ xử lý bất đồng bộ (*async/await*) nhằm tăng hiệu suất xử lý nhiều request đồng thời.
 - * Tối ưu tốc độ phản hồi cho các tác vụ AI và xử lý video.
- **Ưu điểm:**
 - Hiệu năng cao và hỗ trợ xử lý bất đồng bộ.
 - Dễ dàng tích hợp với AI Service và Database Layer.
 - Hỗ trợ tự động sinh tài liệu API thông qua Swagger UI và ReDoc.
 - Kiến trúc module hóa giúp hệ thống dễ mở rộng và bảo trì.

c) Tầng Nghiệp vụ (Service Layer)

- **Mô hình thiết kế:** *Business Logic Layer*.
- **Vai trò:** Là tầng trung tâm xử lý toàn bộ logic nghiệp vụ cốt lõi của hệ thống, đóng vai trò kết nối giữa API Layer, AI Engine và Database Layer.
- **Chức năng chính:**
 - **LPR Service:**
 - * Tiếp nhận dữ liệu hình ảnh hoặc khung hình video từ API Layer.
 - * Tiền xử lý dữ liệu đầu vào trước khi đưa vào mô hình AI.
 - * Gọi mô hình AI để thực hiện nhận diện biển số xe (*License Plate Recognition*).
 - * Chuẩn hóa và trả về kết quả nhận diện cho các phân hệ liên quan.
 - **Traffic & Blacklist Service:**
 - * Tra cứu dữ liệu phương tiện từ cơ sở dữ liệu.
 - * Đối chiếu biển số xe phát hiện được với danh sách đen (*Blacklist*).
 - * Phân tích hành vi vi phạm giao thông và tính toán mức độ vi phạm từ 2 – 10 điểm.
 - * Sinh cảnh báo đối với các phương tiện vi phạm hoặc có trong danh sách theo dõi.
 - **Video Service:**
 - * Quản lý vòng đời của tệp video được tải lên hệ thống.
 - * Tổ chức hàng đợi (*Queue*) cho các tác vụ xử lý video.
 - * Điều phối luồng phân tích dữ liệu giữa các module AI.
 - * Theo dõi trạng thái xử lý: *Pending, Processing, Completed, Failed*.
 - **Logging & Monitoring:**
 - * Ghi log quá trình xử lý nghiệp vụ và lỗi hệ thống.
 - * Hỗ trợ giám sát hiệu năng và truy vết lỗi trong quá trình vận hành.
 - **Data Transformation:**
 - * Chuẩn hóa dữ liệu đầu vào và đầu ra giữa các tầng trong hệ thống.
 - * Giảm sự phụ thuộc trực tiếp giữa các module chức năng.
- **Ưu điểm:**
 - Tách biệt rõ ràng giữa xử lý nghiệp vụ và giao diện API.
 - Dễ dàng mở rộng các module AI hoặc thêm nghiệp vụ mới.
 - Tăng khả năng tái sử dụng mã nguồn và giảm phụ thuộc giữa các thành phần.
 - Hỗ trợ bảo trì và kiểm thử hệ thống hiệu quả hơn.

d) Tầng Truy cập Dữ liệu (Data Access Layer – DAL)

- **Mô hình thiết kế:**

- *Repository Pattern*
- ORM SQLAlchemy

- **Vai trò:** Đóng vai trò là lớp trung gian giữa Service Layer và Database Layer, chịu trách nhiệm quản lý toàn bộ thao tác truy cập dữ liệu của hệ thống.

- **Chức năng chính:**

- **Data Abstraction:**

- * Thực hiện trừu tượng hóa (*Abstraction*) các thao tác làm việc với cơ sở dữ liệu.
- * Tách biệt hoàn toàn logic nghiệp vụ khỏi các truy vấn SQL thuần túy.

- **Repository Management:**

- * Tổ chức các Repository theo từng module chức năng: *User Repository*, *Vehicle Repository*, *Traffic Repository*, *Video Repository*.
- * Cung cấp các phương thức CRUD: *Create*, *Read*, *Update*, *Delete*.

- **ORM Mapping:**

- * Sử dụng SQLAlchemy ORM để ánh xạ (*Mapping*) giữa các bảng dữ liệu và đối tượng Python.
- * Giảm độ phức tạp khi thao tác dữ liệu và tăng tính trực quan trong phát triển hệ thống.

- **Transaction Handling:**

- * Quản lý transaction nhằm đảm bảo tính toàn vẹn dữ liệu (*Data Integrity*).
- * Hỗ trợ cơ chế rollback khi xảy ra lỗi trong quá trình xử lý.

- **Query Optimization:**

- * Tối ưu hiệu suất truy vấn dữ liệu.
- * Hỗ trợ phân trang (*Pagination*), lọc dữ liệu và tìm kiếm nâng cao.

- **Database Independence:**

- * Hỗ trợ khả năng thay đổi hệ quản trị cơ sở dữ liệu trong tương lai mà không ảnh hưởng đến Service Layer.
- * Dễ dàng mở rộng sang các hệ quản trị như PostgreSQL, MySQL hoặc MariaDB.

- **Ưu điểm:**

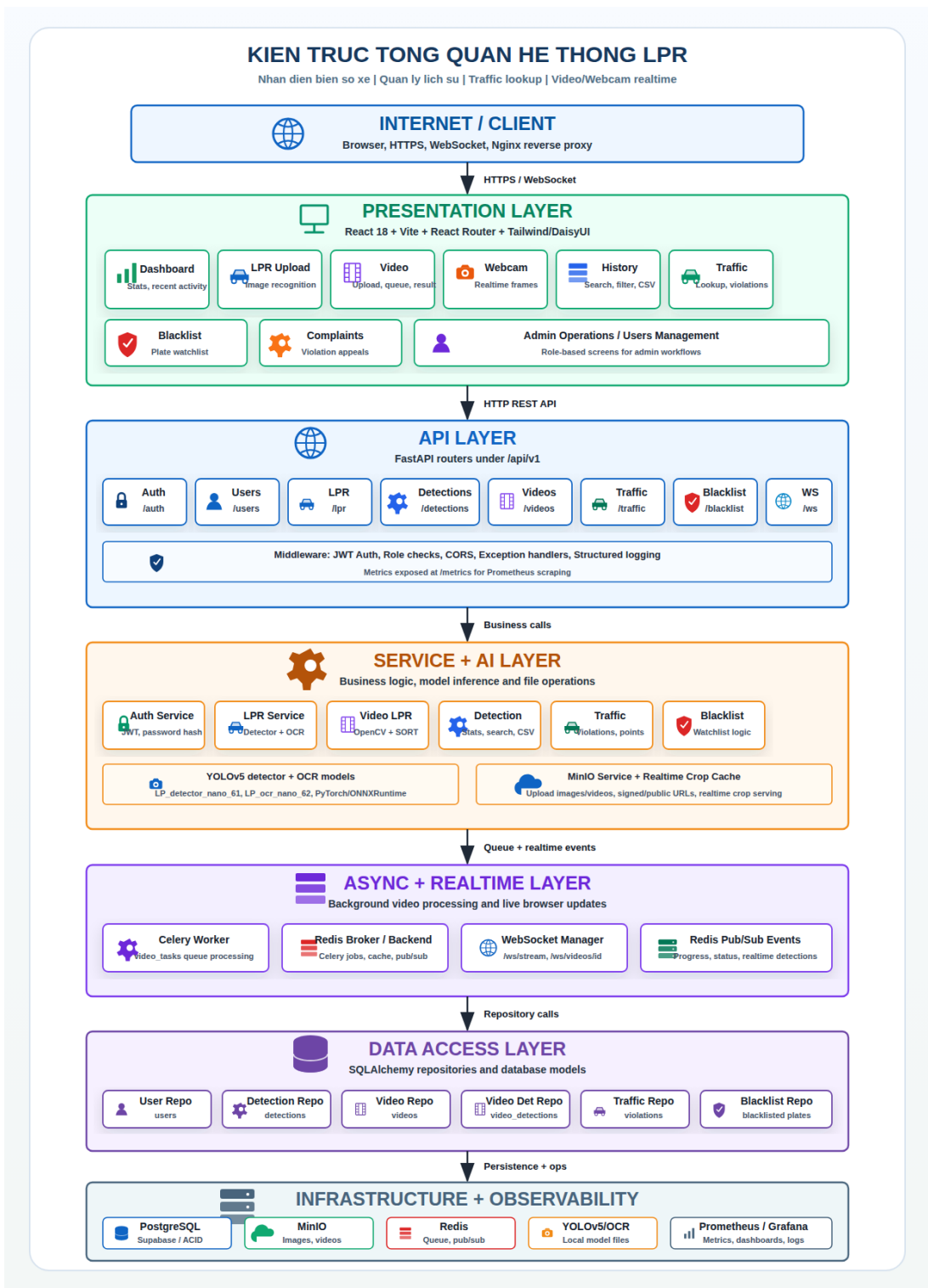
- Tăng khả năng bảo trì và tái sử dụng mã nguồn.
- Giảm sự phụ thuộc giữa nghiệp vụ và cơ sở dữ liệu.
- Dễ dàng kiểm thử (*Unit Test*) thông qua Repository Mocking.
- Hỗ trợ mở rộng và tối ưu hệ thống dữ liệu trong tương lai.

e) **Tầng Hạ tầng và Xử lý Bất đồng bộ**

(Infrastructure & Heavy Processing Microservices)

- **Vai trò:** Quản lý lưu trữ dữ liệu và xử lý các tác vụ nặng theo cơ chế bất đồng bộ (*Asynchronous Processing*) nhằm giảm tải cho API Server.
- **Mục tiêu:**
 - Giải quyết bài toán nghẽn cổ chai (*Bottleneck*) khi xử lý video và dữ liệu thời gian thực.
 - Tăng khả năng mở rộng và hiệu năng hệ thống.
 - Đảm bảo phản hồi nhanh cho người dùng (*Non-blocking UI*).
- **Các thành phần chính:**
 - **PostgreSQL:** Lưu trữ dữ liệu có cấu trúc như tài khoản, lịch sử nhận diện, danh sách đen và dữ liệu vi phạm.
 - **MinIO Object Storage:** Lưu trữ dữ liệu phi cấu trúc gồm ảnh, video và ảnh cắt biến số.
 - **Redis Cache & Queue:** Hoạt động như *Message Broker*, hỗ trợ hàng đợi tác vụ và cơ chế cache.
 - **Celery Worker Microservices:** Xử lý video nền bằng cách bóc tách frame, tiền xử lý ảnh và gọi mô hình AI nhận diện biến số. Các Worker nhận tác vụ từ Redis giúp giảm tải cho API Server chính.
- **Ưu điểm:**
 - Hỗ trợ xử lý song song các tác vụ AI.
 - Tăng khả năng mở rộng hệ thống.
 - Cải thiện hiệu năng và trải nghiệm người dùng.
 - Phù hợp triển khai theo mô hình Microservices và Docker.

2 Hình ảnh tổng quan hệ thống



Hình 1: Kiến trúc tổng thể của hệ thống

V THIẾT KẾ CƠ SỞ DỮ LIỆU

1 Tổng quan cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được xây dựng nhằm hỗ trợ toàn bộ quy trình xử lý vi phạm giao thông thông minh, bao gồm: phát hiện biển số xe từ ảnh/video, tra cứu thông tin chủ phương tiện, ghi nhận vi phạm, xử lý khiếu nại và quản lý danh sách đen phương tiện vi phạm.

Hệ thống được thiết kế theo mô hình cơ sở dữ liệu quan hệ (*Relational Database Model*) nhằm đảm bảo tính toàn vẹn, khả năng mở rộng và hiệu suất xử lý dữ liệu trong môi trường thực tế.

Cơ sở dữ liệu bao gồm 11 bảng chính, được liên kết với nhau thông qua các khóa chính (*Primary Key*) và khóa ngoại (*Foreign Key*). Ngoài ra, hệ thống còn áp dụng các cơ chế kiểm tra dữ liệu, chỉ mục tối ưu truy vấn và các ràng buộc toàn vẹn nhằm đảm bảo độ chính xác và ổn định khi vận hành.

2 Các nhóm thực thể chính

a) Người dùng và xác thực

- **users:** Lưu trữ thông tin tài khoản người dùng của hệ thống, bao gồm: Tên đăng nhập, Mật khẩu đã mã hóa, Vai trò và Trạng thái hoạt động. Đây là bảng trung tâm kết nối với các nghiệp vụ khác.

Bảng 11: Cấu trúc bảng users

Thuộc tính	Mô tả
id	Khóa chính của bảng người dùng
username	Tên đăng nhập
password	Mật khẩu đã được mã hóa (hashed password)
role	Vai trò người dùng (user/admin)
is_active	Trạng thái hoạt động của tài khoản
created_at	Thời gian tạo tài khoản

b) **Phương tiện và chủ sở hữu**

- **owners:** Lưu thông tin chủ sở hữu phương tiện bao gồm: Họ tên, Số CMND/CCCD (duy nhất), Số điện thoại và Địa chỉ.

Bảng 12: Cấu trúc bảng owners

Thuộc tính	Mô tả
id	Khóa chính
full_name	Họ tên chủ phương tiện
citizen_id	Số CMND/CCCD (duy nhất)
phone_number	Số điện thoại
address	Địa chỉ
created_at	Thời gian tạo dữ liệu

- **vehicles:** Lưu thông tin phương tiện (Biển số xe duy nhất, Loại xe, Nhân hiệu, Màu sắc, Ngày đăng ký). Bảng này liên kết với bảng **owners** thông qua khóa ngoại *owner_id* (ON DELETE SET NULL). Ngoài ra còn chứa cờ *is_blacklist* và lý do kèm theo.

Bảng 13: Cấu trúc bảng vehicles

Thuộc tính	Mô tả
id	Khóa chính
plate_number	Biển số xe (duy nhất)
owner_id	Khóa ngoại liên kết owners
vehicle_type	Loại phương tiện
brand	Nhãn hiệu xe
color	Màu sắc
registration_date	Ngày đăng ký phương tiện
is_blacklist	Trạng thái danh sách đen (0/1)
blacklist_reason	Lý do đưa vào blacklist
created_at	Thời gian tạo dữ liệu

c) **Phát hiện và ghi nhận biển số**

- **detection_histories**: Lưu lịch sử phát hiện biển số từ ảnh, bao gồm thông tin người thực hiện, biển số, độ tin cậy (*confidence*), ảnh đầu vào và trạng thái thuộc danh sách đen.

Bảng 14: Cấu trúc bảng detection_histories

Thuộc tính	Mô tả
id	Khóa chính
user_id	Người thực hiện phát hiện
plate_number	Biển số phát hiện được
confidence	Độ tin cậy của mô hình AI
image_path	Đường dẫn ảnh
is_blacklisted	Có nằm trong blacklist hay không
detected_at	Thời gian phát hiện

- **uploaded_videos**: Quản lý video được tải lên hệ thống cùng với đường dẫn video gốc, video sau xử lý và trạng thái xử lý (*processing, completed, ...*).

Bảng 15: Cấu trúc bảng uploaded_videos

Thuộc tính	Mô tả
id	Khóa chính
user_id	Người tải video lên
original_video_path	Đường dẫn video gốc
processed_video_path	Đường dẫn video đã xử lý
status	Trạng thái xử lý video
created_at	Thời gian tải lên

- **detections**: Lưu kết quả phát hiện biển số từ video gồm biển số xe, độ tin cậy, thời điểm xuất hiện, khung hình và tọa độ vùng nhận diện (*bounding box*) dạng JSON.

Bảng 16: Cấu trúc bảng detections

Thuộc tính	Mô tả
id	Khóa chính
uploaded_video_id	Liên kết video đã tải lên
plate_number	Biển số được phát hiện
confidence	Độ tin cậy nhận diện
frame_time	Thời điểm xuất hiện trong video
frame_path	Đường dẫn khung hình
bounding_box	Tọa độ vùng nhận diện dạng JSON
created_at	Thời gian tạo dữ liệu

- **video_detections**: Lưu thông tin phát hiện chi tiết hơn phục vụ kỹ thuật (Số khung hình, Tọa độ bounding box, Kích thước khung hình).

Bảng 17: Cấu trúc bảng video_detections

Thuộc tính	Mô tả
id	Khóa chính
uploaded_video_id	Liên kết video
frame_number	Số thứ tự khung hình
bounding_box	Tọa độ bounding box
frame_width	Chiều rộng khung hình
frame_height	Chiều cao khung hình
created_at	Thời gian tạo dữ liệu

- **video_plate_detections**: Có chức năng tương tự **video_detections** nhưng sử dụng cấu trúc dữ liệu khác phục vụ quá trình tối ưu hoặc chạy song song các phiên bản xử lý video.

Bảng 18: Cấu trúc bảng video_plate_detections

Thuộc tính	Mô tả
id	Khóa chính
uploaded_video_id	Liên kết video
plate_number	Biển số phát hiện được
frame_number	Số khung hình
bounding_box	Tọa độ nhận diện
confidence	Độ tin cậy nhận diện
created_at	Thời gian tạo dữ liệu

d) Quản lý vi phạm

- **violations**: Lưu thông tin các hành vi vi phạm giao thông (Loại vi phạm, Số tiền phạt, Điểm trừ, Trạng thái xử lý). Bảng liên kết với **vehicles** thông qua khóa ngoại *vehicle_id* (ON DELETE CASCADE).

Bảng 19: Cấu trúc bảng violations

Thuộc tính	Mô tả
id	Khóa chính
vehicle_id	Liên kết phương tiện vi phạm
violation_type	Loại vi phạm
fine_amount	Số tiền phạt
penalty_points	Điểm trừ
status	Trạng thái xử lý
issued_at	Thời gian lập biên bản

e) **Danh sách đen**

- **blacklisted_plates**: Lưu các biển số xe bị đưa vào danh sách đen bao gồm biển số, lý do và người tạo bản ghi. Trường *created_by* tham chiếu đến bảng **users** (ON DELETE SET NULL).

Bảng 20: Cấu trúc bảng blacklisted_plates

Thuộc tính	Mô tả
id	Khóa chính
plate_number	Biển số nằm trong blacklist
reason	Lý do đưa vào blacklist
created_by	Người tạo bản ghi
created_at	Thời gian tạo dữ liệu

f) **Khiếu nại vi phạm**

- **complaints**: Lưu thông tin khiếu nại liên quan đến các vi phạm giao thông (Thông tin người khiếu nại, Nội dung khiếu nại, Bằng chứng đính kèm, Trạng thái xử lý). Bảng liên kết với **violations** và **users**.

Bảng 21: Cấu trúc bảng complaints

Thuộc tính	Mô tả
id	Khóa chính
violation_id	Liên kết vi phạm
user_id	Người dùng gửi khiếu nại
plate_number	Biển số liên quan
complainant_name	Họ tên người khiếu nại
citizen_id	CMND/CCCD người khiếu nại
phone_number	Số điện thoại
address	Địa chỉ
reason	Nội dung khiếu nại
evidence_path	Đường dẫn bằng chứng
status	Trạng thái xử lý
created_at	Thời gian gửi khiếu nại

3 Các đặc điểm nổi bật trong thiết kế

- **Tối ưu hóa truy vấn:** Hệ thống sử dụng nhiều chỉ mục (*Index*) trên các cột thường xuyên truy vấn như *plate_number*, *user_id*, và *uploaded_video_id*, giúp tăng tốc độ tìm kiếm dữ liệu.
- **Ràng buộc toàn vẹn linh hoạt:** Các mối quan hệ được ràng buộc chặt chẽ thông qua các cơ chế phản ứng phù hợp với nghiệp vụ như `ON DELETE CASCADE` hoặc `ON DELETE SET NULL`.
- **Kiểm soát miền giá trị:** Áp dụng ràng buộc kiểm tra (*CHECK Constraint*) cho trường *is_blacklist*, chỉ cho phép nhận giá trị 0 (Không nằm trong danh sách đen) hoặc 1 (Nằm trong danh sách đen).
- **Tự động hóa thời gian:** Các trường thời gian (*created_at*, *detected_at*, *issued_at*) đều được thiết lập giá trị mặc định bằng `CURRENT_TIMESTAMP` hoặc `now()` nhằm tự động ghi nhận chính xác thời điểm phát sinh dữ liệu.

4 Đánh giá thiết kế

Nhìn chung, thiết kế cơ sở dữ liệu đáp ứng tốt yêu cầu của một hệ thống giám sát giao thông thông minh với khả năng quản lý dữ liệu tập trung, hỗ trợ xử lý ảnh/video hiệu quả, đảm bảo tính toàn vẹn và dễ dàng mở rộng trong tương lai.

Tuy nhiên, việc tồn tại đồng thời các bảng **detections**, **video_detections**,... có thể gây dư thừa dữ liệu và khó khăn trong quá trình bảo trì. Vì vậy, trong các phiên bản tiếp theo, nhóm phát triển nên xem xét chuẩn hóa và hợp nhất các bảng này nhằm tối ưu hóa cấu trúc hệ thống.

VI THIẾT KẾ PIPELINE AI

1 Tổng quan Pipeline AI

Hệ thống AI nhận diện biển số xe (*License Plate Recognition - LPR*) được xây dựng theo kiến trúc hai giai đoạn (*Two-stage Architecture*) sử dụng mô hình YOLOv5.

Pipeline AI hỗ trợ:

- Nhận diện biển số từ ảnh đơn cấu trúc;
- Nhận diện thời gian thực qua luồng webcam;
- Xử lý và trích xuất dữ liệu biển số từ video;
- Chuẩn hóa dữ liệu biển số theo định dạng quy chuẩn Việt Nam.

Hệ thống hoạt động tuần tự theo quy trình tổng quát dưới đây:

Input → Detection → OCR → Post-processing → Result

2 Kiến trúc mô hình AI

Hệ thống phân rã bài toán nhận diện thành hai mô hình chuyên biệt đảm nhiệm các vai trò độc lập:

Bảng 22: Các mô hình AI sử dụng trong hệ thống

Mô hình	Công nghệ	Chức năng
LP_detector_nano_61	YOLOv5 + ONNX	Phát hiện và định vị vùng chứa biển số xe
LP_ocr_nano_62	YOLOv5 + ONNX	Phân tách và nhận diện các ký tự trên biển số

Việc áp dụng kiến trúc *Two-stage* (Phát hiện vị trí biển số trước, sau đó mới tiến hành nhận diện ký tự OCR) giúp cải thiện đáng kể độ chính xác của hệ thống khi phải xử lý các trường hợp vật thể biển số có kích thước nhỏ, mờ do chuyển động, hoặc khi xuất hiện nhiều phương tiện đồng thời trong cùng một khung hình.

3 Pipeline xử lý ảnh

Khi nhận được yêu cầu xử lý hình ảnh tĩnh từ người dùng, pipeline AI sẽ thực thi tuần tự các bước sau:

1. Giải mã dữ liệu ảnh đầu vào từ request upload;
2. Tiền xử lý ảnh và chuẩn hóa cấu trúc về không gian màu RGB;
3. Sử dụng mô hình YOLOv5 Detector để xác định tọa độ vùng chứa biển số;
4. Thực hiện cắt vùng ảnh chứa biển số (*Crop*);
5. Áp dụng mô hình OCR để nhận diện từng ký tự riêng lẻ;
6. Ghép chuỗi ký tự và chuẩn hóa cấu trúc định dạng theo quy định biển số xe Việt Nam;
7. Đóng gói và trả kết quả về phía client dưới dạng cấu trúc dữ liệu JSON.

Bảng 23: Quy trình nhận diện biển số từ hình ảnh

Bước xử lý	Mô tả chi tiết
Image Upload	Hệ thống tiếp nhận tập tin ảnh được tải lên từ phía người dùng.
Plate Detection	Mô hình AI quét toàn bộ ảnh và khoanh vùng vị trí biển số.
Plate Crop	Trích xuất riêng phần ảnh chứa biển số ra khỏi khung hình gốc.
OCR Recognition	Phân tích vùng biên và nhận diện giá trị văn bản của từng ký tự.
Normalize Plate	Chuyển đổi chuỗi ký tự thu được về đúng quy chuẩn cấu trúc biển số VN.
JSON Response	Trả về payload dữ liệu chứa biển số, tọa độ và độ tin cậy.

4 Pipeline xử lý video

Đối với định dạng video dung lượng lớn, hệ thống triển khai cơ chế xử lý bất đồng bộ (*Asynchronous Processing*) thông qua nền tảng Celery Worker nhằm giảm thiểu tối đa hiện tượng nghẽn mạch (blocking) tại API Server.

1. Người dùng tải video lên; hệ thống tiếp nhận và lưu trữ tệp tin gốc vào MinIO Storage;
2. Hệ thống đẩy một tác vụ xử lý video mới vào hàng đợi Redis Queue;
3. Celery Worker nhận tác vụ, tiến hành đọc luồng video và phân tách thành các khung hình (*Frames*);
4. Core AI thực hiện vòng lặp phát hiện và nhận diện biển số trên từng khung hình được trích xuất;
5. Toàn bộ thông tin nhận diện được ghi nhận trực tiếp vào cơ sở dữ liệu quan hệ;
6. Tiến trình xử lý tổng thể được cập nhật liên tục về phía giao diện người dùng theo thời gian thực thông qua kết nối WebSocket.

Bảng 24: Các thành phần cấu tạo trong pipeline xử lý video

Thành phần	Chức năng trong hệ thống
MinIO Storage	Lưu trữ và quản lý tập trung các tệp tin video gốc an toàn.
Redis Queue	Đóng vai trò làm broker trung chuyển, điều phối hàng đợi tác vụ.
Celery Worker	Thành phần nền chịu trách nhiệm giải mã video và trích xuất từng frame.
YOLOv5 Detector	Thực hiện tác vụ suy luận phân tích vị trí biển số trên từng frame ảnh.
OCR Model	Xử lý nhận dạng ký tự đối với các vùng biển số được tìm thấy.
WebSocket	Kênh truyền dẫn hai chiều, đẩy trạng thái và tiến trình hoàn thành realtime.

5 Pipeline Webcam Realtime

Nhằm đáp ứng nhu cầu kiểm soát luồng giao thông tại các trạm barrier trực tiếp, hệ thống cung cấp pipeline xử lý luồng dữ liệu thời gian thực từ Webcam hoặc IP Camera Streaming.

Luồng xử lý tối ưu bao gồm:

1. Client-side bắt luồng video từ thiết bị ghi hình và gửi liên tục các frame ảnh về Backend;
2. API Gateway phân phối frame đến phân hệ AI để thực hiện nhận diện tức thời;
3. Bộ xử lý OCR bóc tách chuỗi ký tự và đối chiếu nhanh với danh sách nghiệp vụ;

4. Phản hồi kết quả nhận diện về màn hình giám sát với độ trễ tối thiểu (gần như tức thời).

Bảng 25: Quy trình phân tích dữ liệu Webcam Realtime

Bước xử lý	Mô tả chi tiết tác vụ
Realtime Frame Input	Thu nhận liên tục luồng frame ảnh thô từ camera giám sát.
YOLO Detection	Định vị nhanh vị trí biển số xuất hiện trong frame hiện tại.
OCR Processing	Biên dịch ma trận điểm ảnh ký tự thành chuỗi text tương ứng.
Realtime Response	Stream dữ liệu kết quả về giao diện để hiển thị tức thì cho giám sát viên.

6 Các kỹ thuật tối ưu hiệu năng

Để đảm bảo hệ thống đạt được hiệu suất cao nhất và có khả năng phục vụ nhiều luồng xử lý đồng thời trong môi trường thực tế, các kỹ thuật tối ưu kiến trúc đã được áp dụng chặt chẽ:

- **ONNX Runtime Acceleration:** Chuyển đổi định dạng mô hình nguyên bản sang định dạng ONNX giúp tối ưu hóa đồ thị tính toán và tăng tốc độ suy luận (*Inference*).
- **Dynamic Hardware Selection:** Hệ thống tự động phát hiện và cấu hình sử dụng phần cứng tăng tốc phần mềm (GPU CUDA) nếu khả dụng, ngược lại sẽ tối ưu luồng chạy đa nhân trên CPU.
- **Singleton Pattern Architecture:** Đảm bảo các mô hình học máy dung lượng lớn chỉ được khởi tạo và tải vào bộ nhớ duy nhất một lần khi khởi động hệ thống, tránh lãng phí tài nguyên RAM/VRAM.
- **Thread-safe Control:** Áp dụng cơ chế khóa dữ liệu (*Lock Mechanism*) tại các tiến trình xử lý đa luồng nhằm ngăn chặn triệt để hiện tượng xung đột tài nguyên (*Race Condition*).

Bảng 26: Tổng hợp các giải pháp và mục tiêu tối ưu hệ thống AI

Giải pháp áp dụng	Mục tiêu tối ưu hóa hành vi
ONNX Runtime	Rút ngắn thời gian suy luận của các mô hình mạng nơ-ron.
CUDA GPU Support	Tận dụng tối đa sức mạnh tính toán song song của phần cứng chuyên dụng.
Thread Lock	Bảo vệ tính toàn vẹn dữ liệu và độ ổn định khi xử lý đa luồng.
Celery Queue	Giải phóng tài nguyên API, xử lý bất đồng bộ các luồng dữ liệu nặng.
WebSocket Streaming	Giảm thiểu băng thông luồng kéo (polling), tối ưu hóa trải nghiệm realtime.

7 Đánh giá tổng kết Pipeline AI

Nhìn chung, giải pháp thiết kế Pipeline AI của hệ thống đáp ứng toàn diện các tiêu chí khắt khe của một hệ thống quản lý đô thị thông minh:

- Đạt độ chính xác cao đối với các biến thể cấu trúc biển số xe tại Việt Nam (biển ngắn, biển dài, biển nền xanh/trắng/vàng);
- Linh hoạt trong việc chuyển đổi và thích ứng giữa 3 chế độ dữ liệu: Ảnh tĩnh, Video lưu trữ và Luồng trực tiếp (Webcam);
- Kiến trúc mở, tách biệt phần core xử lý AI với API server giúp hệ thống dễ dàng thực hiện scale-up phần cứng hoặc cập nhật các phiên bản mô hình AI tiên tiến hơn trong tương lai mà không làm gián đoạn vận hành.

VII THIẾT KẾ API

1 Tổng quan hệ thống API

Hệ thống sử dụng kiến trúc RESTful API nhằm hỗ trợ giao tiếp giữa Frontend, Backend và các dịch vụ xử lý trí tuệ nhân tạo (*AI Services*). Toàn bộ API được xây dựng theo chuẩn REST, sử dụng dữ liệu định dạng JSON để trao đổi thông tin giữa các thành phần trong hệ thống.

Tất cả endpoint đều sử dụng tiền tố:

/api/v1

Hệ thống áp dụng cơ chế xác thực bằng JWT Bearer Token nhằm đảm bảo tính bảo mật và kiểm soát quyền truy cập của người dùng. Sau khi đăng nhập thành công, người dùng sẽ nhận được Access Token để sử dụng cho các request yêu cầu xác thực.

Ngoài ra, hệ thống còn hỗ trợ:

- Phân quyền người dùng (*User/Admin*);
- Giao tiếp thời gian thực bằng WebSocket;
- Hỗ trợ xử lý bất đồng bộ cho video;
- Khả năng mở rộng API theo phiên bản.

2 Authentication API

Nhóm API xác thực hỗ trợ đăng ký, đăng nhập và kiểm tra thông tin người dùng hiện tại.

Bảng 27: Danh sách Authentication API

Method	Endpoint	Auth	Mô tả
POST	/api/v1/auth/register	Không	Đăng ký tài khoản mới
POST	/api/v1/auth/login	Không	Đăng nhập và trả về JWT Token
GET	/api/v1/auth/me	Có	Lấy thông tin người dùng hiện tại

3 LPR Recognition API

Nhóm API nhận diện biển số xe (*License Plate Recognition*) hỗ trợ xử lý ảnh và luồng dữ liệu thời gian thực.

Bảng 28: Danh sách LPR Recognition API

Method	Endpoint	Auth	Mô tả
POST	/api/v1/lpr/recognize	Có	Nhận diện biển số từ ảnh
POST	/api/v1/lpr/realtime-frame	Có	Nhận diện biển số theo thời gian thực

4 Detection History API

Nhóm API quản lý lịch sử phát hiện biển số và hỗ trợ thống kê dữ liệu.

Bảng 29: Danh sách Detection History API

Method	Endpoint	Auth	Mô tả
GET	/api/v1/detections	Có	Lấy danh sách phát hiện
POST	/api/v1/detections	Có	Tạo bản ghi phát hiện mới
GET	/api/v1/detections/search	Có	Tìm kiếm và phân trang dữ liệu
GET	/api/v1/detections/export	Có	Xuất dữ liệu dưới dạng CSV
GET	/api/v1/detections/stats	Có	Thống kê dữ liệu phát hiện

5 Video Processing API

Nhóm API xử lý video hỗ trợ tải lên video và đưa video vào hàng đợi xử lý bất đồng bộ.

Bảng 30: Danh sách Video API

Method	Endpoint	Auth	Mô tả
GET	/api/v1/videos	Có	Lấy danh sách video đã tải lên
POST	/api/v1/videos	Có	Upload video lên hệ thống
POST	/api/v1/videos/{id}/queue	Có	Đưa video vào hàng đợi xử lý

6 Traffic Violation API

Nhóm API quản lý vi phạm giao thông, tra cứu biển số và xử lý khiếu nại.

Bảng 31: Danh sách Traffic API

Method	Endpoint	Auth	Mô tả
GET	/api/v1/traffic/lookup/{plate}	Có	Tra cứu biển số và điểm vi phạm
GET	/api/v1/traffic/violated-plates	Có	Danh sách biển số đang vi phạm
POST	/api/v1/traffic/complaints	Có	Gửi khiếu nại vi phạm
POST	/api/v1/traffic/violations	Admin	Tạo vi phạm mới
PUT	/api/v1/traffic/complaints/{id}/status	Admin	Duyệt hoặc cập nhật trạng thái khiếu nại
PUT	/api/v1/traffic/violations/{id}	Admin	Chỉnh sửa thông tin vi phạm

7 Blacklist API

Nhóm API quản lý danh sách đen các phương tiện vi phạm.

Bảng 32: Danh sách Blacklist API

Method	Endpoint	Auth	Mô tả
GET	/api/v1/blacklist	Có	Lấy danh sách blacklist
POST	/api/v1/blacklist	Admin	Thêm biển số vào blacklist
DELETE	/api/v1/blacklist/{id}	Admin	Xóa biển số khỏi blacklist

8 User Management API

Nhóm API quản lý người dùng, chỉ dành cho quản trị viên hệ thống.

Bảng 33: Danh sách User Management API

Method	Endpoint	Auth	Mô tả
GET	/api/v1/users	Admin	Lấy danh sách người dùng
POST	/api/v1/users	Admin	Tạo tài khoản người dùng mới
DELETE	/api/v1/users/{id}	Admin	Xóa tài khoản người dùng

9 WebSocket API

Hệ thống hỗ trợ giao tiếp thời gian thực thông qua WebSocket nhằm cập nhật tiến trình xử lý video và sự kiện phát hiện biển số ngay lập tức.

Bảng 34: Danh sách WebSocket Endpoint

Endpoint	Mô tả
/api/v1/ws/stream	Truyền sự kiện thời gian thực cho xử lý video

10 Đánh giá thiết kế API

Thiết kế API của hệ thống đáp ứng tốt các yêu cầu của một hệ thống giám sát giao thông thông minh hiện đại:

- Tuân thủ chuẩn RESTful API;
- Hỗ trợ xác thực và phân quyền bằng JWT;
- Có khả năng mở rộng theo phiên bản API;
- Hỗ trợ xử lý dữ liệu thời gian thực bằng WebSocket;
- Tối ưu cho xử lý bất đồng bộ và AI inference.

Bên cạnh đó, việc tách riêng các nhóm API theo từng nghiệp vụ giúp hệ thống dễ bảo trì, dễ mở rộng và thuận tiện cho việc tích hợp với các hệ thống bên ngoài trong tương lai.

VIII THIẾT KẾ GIAO DIỆN

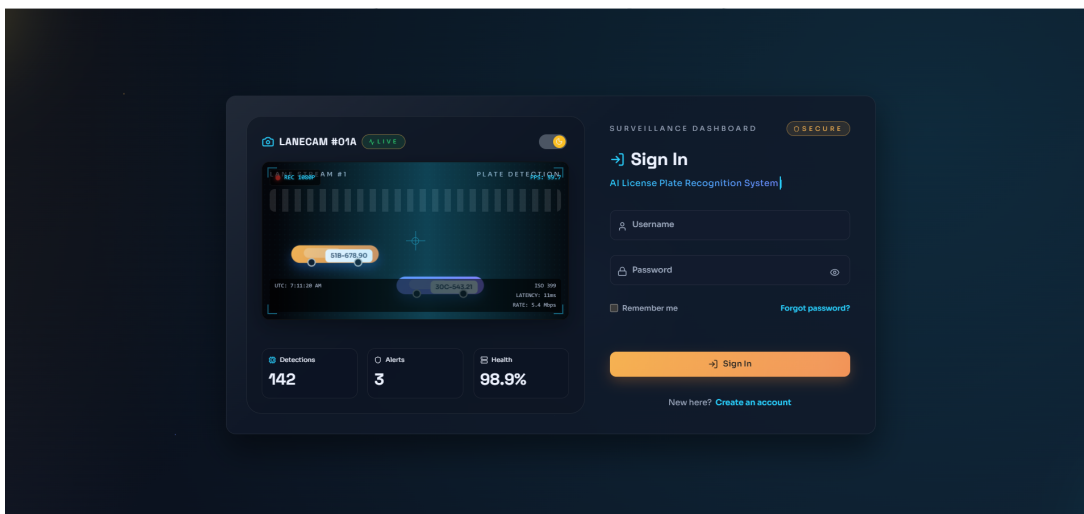
Hệ thống hỗ trợ cả hai chế độ giao diện **Dark Mode** (Chế độ tối) và **Light Mode** (Chế độ sáng) nhằm nâng cao trải nghiệm người dùng trong nhiều điều kiện sử dụng khác nhau:

- + **Chế độ tối:** Sử dụng tông màu xanh đậm kết hợp hiệu ứng ánh sáng hiện đại, tạo cảm giác chuyên nghiệp và phù hợp với các hệ thống giám sát thông minh.
- + **Chế độ sáng:** Mang lại giao diện trực quan, dễ quan sát và thuận tiện khi sử dụng trong môi trường có cường độ ánh sáng cao.

Việc tích hợp hai chế độ hiển thị không chỉ tăng tính thẩm mỹ mà còn giúp giảm mỏi mắt, hỗ trợ cá nhân hóa trải nghiệm và nâng cao tính thân thiện của hệ thống đối với người dùng.

1 Giao diện đăng nhập

Giao diện đăng nhập được thiết kế nhằm hỗ trợ người dùng truy cập hệ thống một cách nhanh chóng và thuận tiện. Màn hình cung cấp các chức năng cơ bản như nhập tên đăng nhập, mật khẩu, ghi nhớ phiên đăng nhập và khôi phục mật khẩu khi cần thiết. Bố cục giao diện được sắp xếp trực quan, giúp người dùng dễ dàng thao tác và sử dụng.

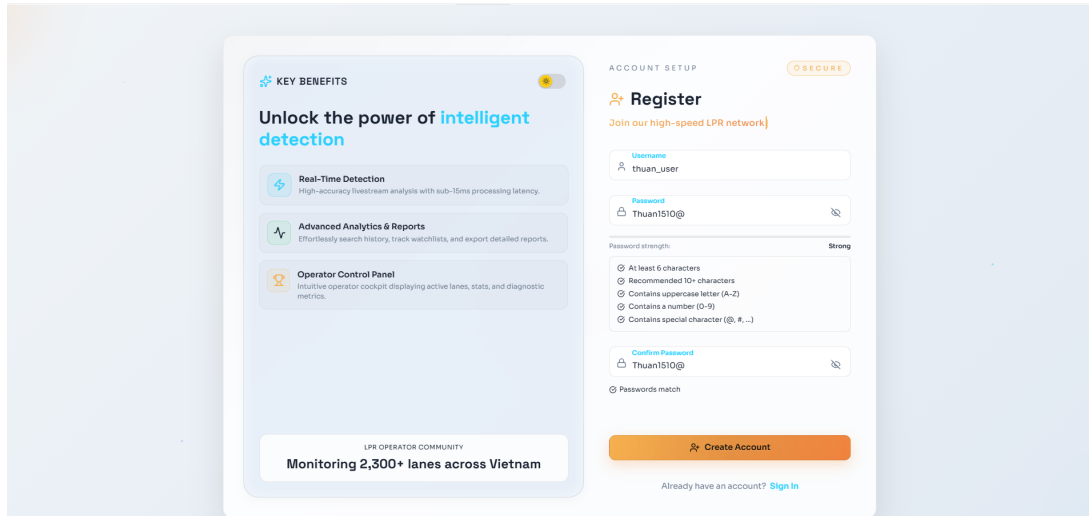


Hình 2: Giao diện đăng nhập chế độ tối

Ngoài chức năng xác thực, giao diện còn hiển thị thông tin tổng quan về hệ thống nhận diện biển số xe tự động, góp phần tăng tính chuyên nghiệp và tạo ấn tượng trực quan cho người dùng khi truy cập hệ thống.

2 Giao diện đăng ký

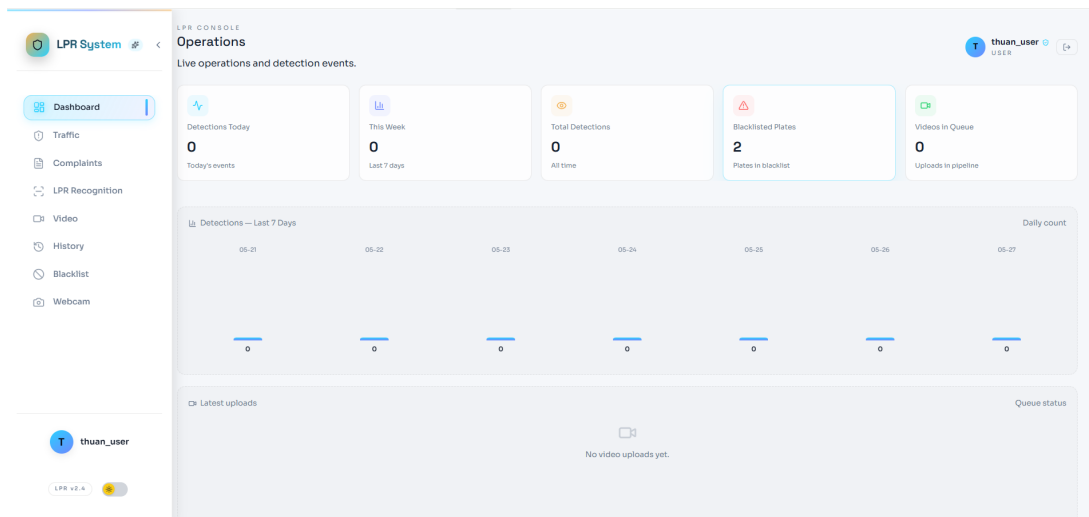
Giao diện đăng ký cho phép người dùng tạo tài khoản mới để sử dụng hệ thống. Các trường thông tin được bố trí rõ ràng và tối giản, bao gồm tên đăng nhập, mật khẩu và xác nhận mật khẩu nhằm đảm bảo tính chính xác trong quá trình đăng ký.



Hình 3: Giao diện đăng kí chế độ sáng

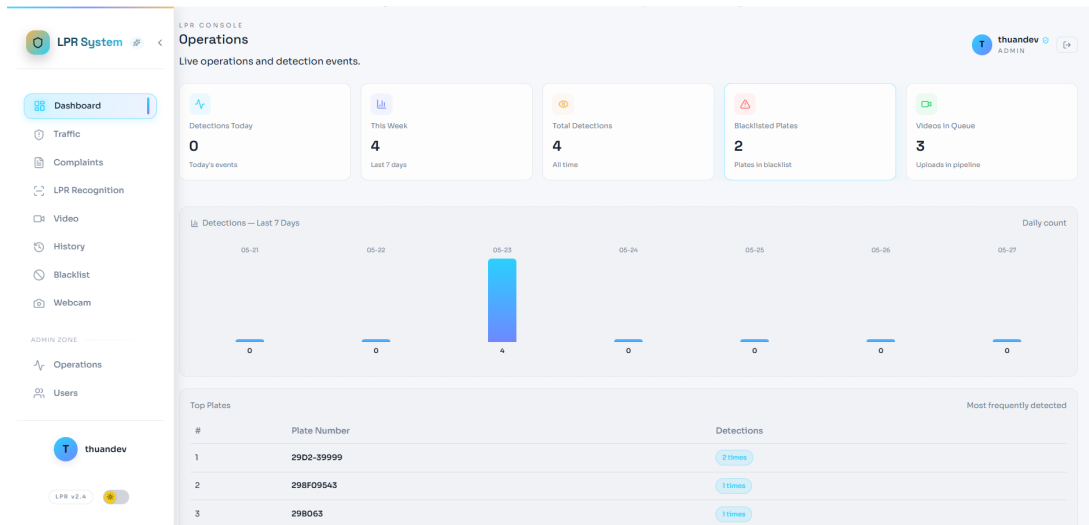
3 Giao diện màn hình chính

Đây là giao diện sẽ thể hiện các thống kê, thông số về lượng ảnh/video đã được nhận diện. Với mỗi quyền User và Admin sẽ có vài điểm giống và khác nhau ở các chức năng trong sidebar. Để biết quyền của người dùng thì có thể nhìn qua góc bên trên phải sẽ thể hiện quyền hoặc góc dưới sidebar.



Hình 4: Giao diện dashboard của user

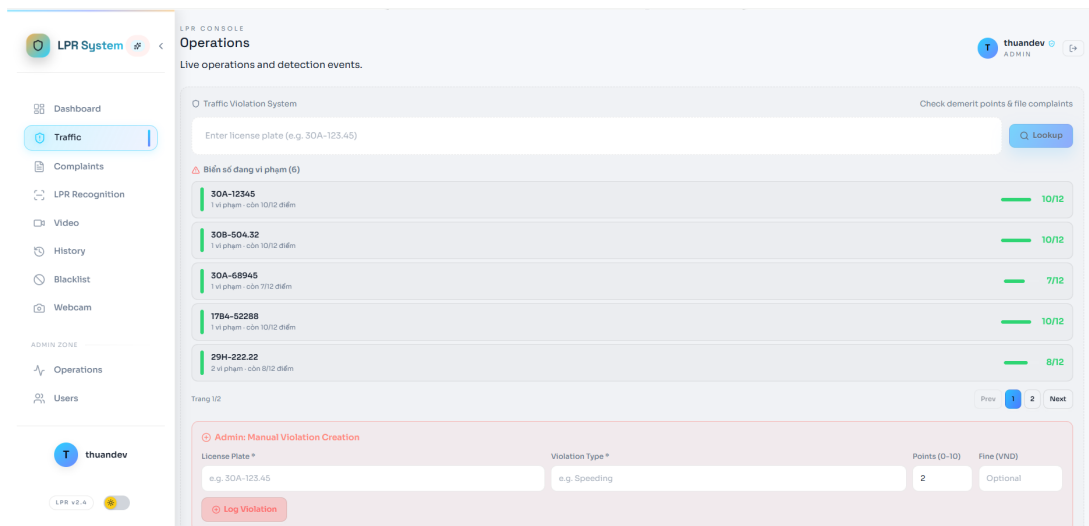
Riêng giao diện của admin, sẽ gồm có các chức năng của user và được thêm các quyền khác như: quản lý lỗi vi phạm của người dùng, quản lý blacklist, quản lý danh sách người dùng,...



Hình 5: Giao diện dashboard của admin

4 Giao diện tìm kiếm lỗi vi phạm

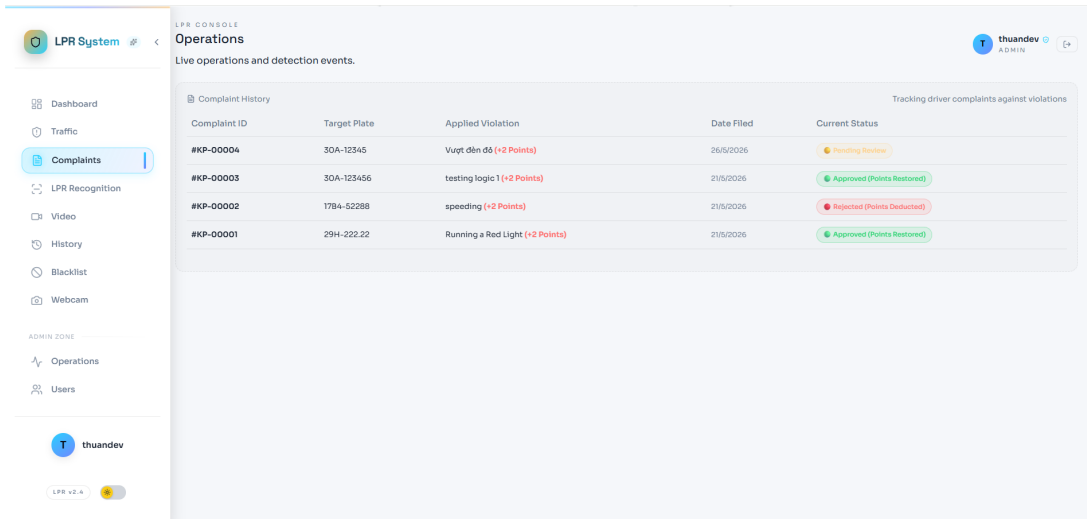
Đây là giao diện mà người dùng muốn kiểm tra thông tin về vấn đề vi phạm giao thông. Người dùng có thể xem được trước các danh sách đang bị vi phạm hoặc có thể tìm kiếm thông tin của bản thân trên thanh searchBar. User có thể khiếu nại nếu đang bị vi phạm hoặc nộp phạt. Điểm khác biệt của Admin/User là user sẽ không thấy được **Manual Violation Creation**, chỉ có Admin mới có thể thực hiện được thao tác này.



Hình 6: Giao diện tìm kiếm lỗi vi phạm của admin

5 Giao diện xem lịch sử tạo yêu cầu

Giao diện này sẽ hiển thị lịch sử và trạng thái khiếu nại/nộp phạt của User, sau khi thao tác trong traffic. Lịch sử của User chỉ hiển thị của từng cá nhân, riêng quyền Admin sẽ hiển thị tất cả danh sách.



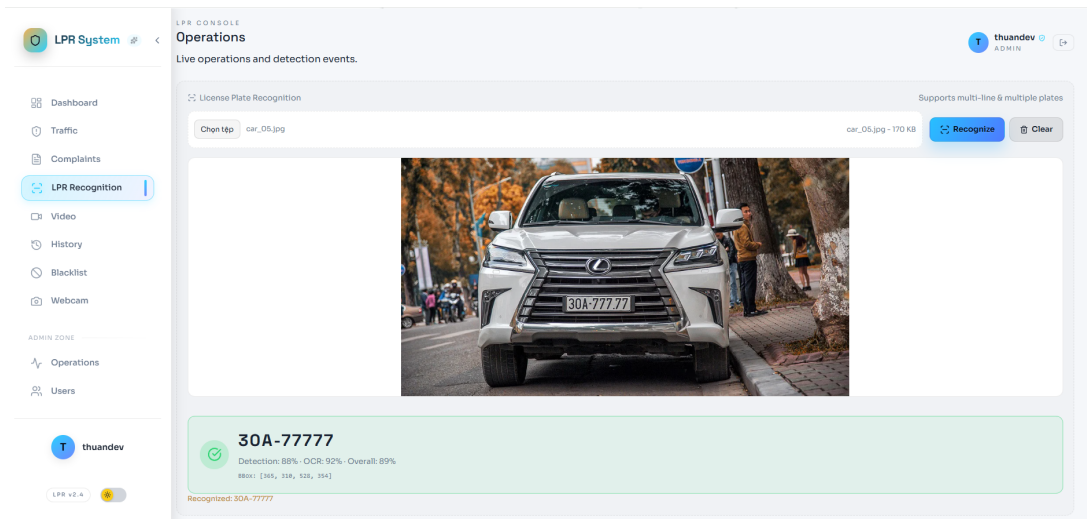
The screenshot shows the 'Operations' section of the LPR System. It features a sidebar with navigation options like Dashboard, Traffic, Complaints, LPR Recognition, Video, History, Blacklist, Webcam, and Admin Zone. The main content area displays a 'Complaint History' table with columns for Complaint ID, Target Plate, Applied Violation, Date Filed, and Current Status. The table lists four entries with various violations like 'Vượt đèn đỏ', 'testing logic 1', 'speeding', and 'Running a Red Light', each with a corresponding status like 'Pending Review', 'Approved (Points Restored)', or 'Rejected (Points Deducted)'.

Complaint ID	Target Plate	Applied Violation	Date Filed	Current Status
#KP-00004	30A-12345	Vượt đèn đỏ (+2 Points)	26/5/2026	Pending Review
#KP-00003	30A-123456	testing logic 1 (+2 Points)	21/5/2026	Approved (Points Restored)
#KP-00002	1784-52288	speeding (+2 Points)	21/5/2026	Rejected (Points Deducted)
#KP-00001	29H-222.22	Running a Red Light (+2 Points)	21/5/2026	Approved (Points Restored)

Hình 7: Giao diện lịch sử tạo yêu cầu

6 Giao diện nhận diện biển số bằng ảnh

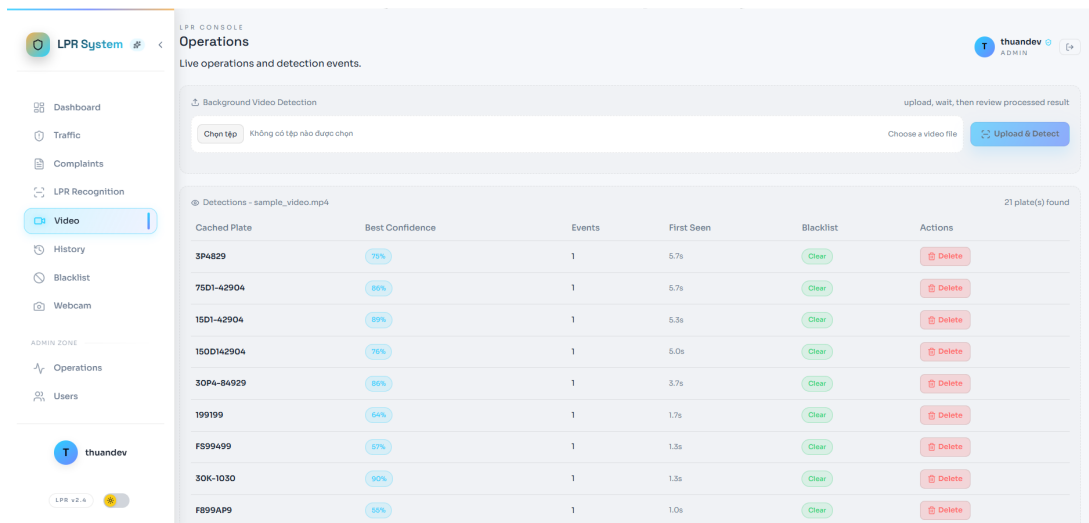
Người dùng có thể upload ảnh để hệ thống có thể xử lý và đưa ra kết quả sau khi detection và recognition. Kết quả sẽ được lưu trong mục history (ở mục sau)



Hình 8: Giao diện upload ảnh

7 Giao diện nhân diện biển số bằng video

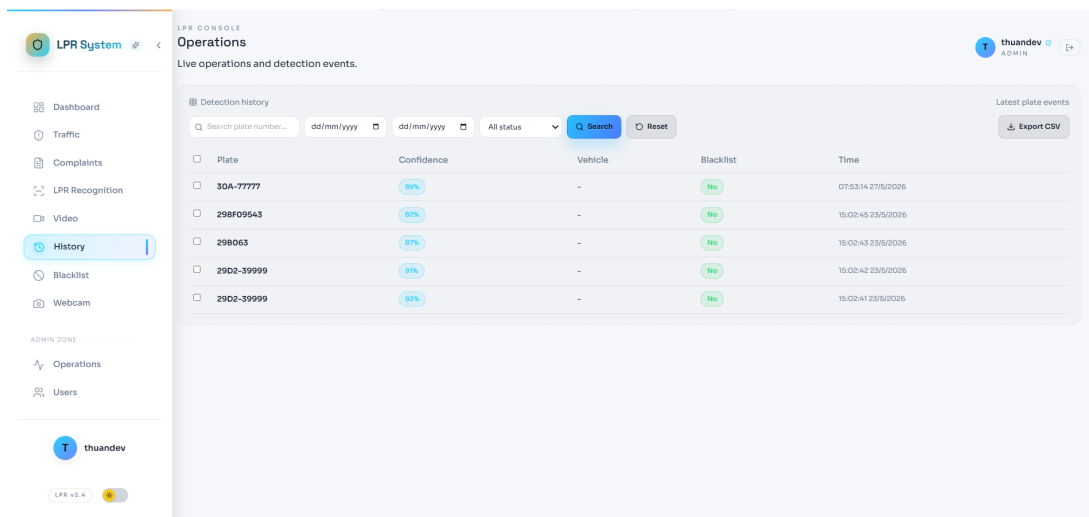
Người dùng upload video mong muốn, sau đó video được xử lý. Kết quả trả về sẽ được hiển thị ở dưới. Đồng thời lịch sử upload video được lưu tại mục này và có thể xem lại video sau khi hệ thống xử lý xong.



Hình 9: Giao diện upload video

8 Giao diện lịch sử

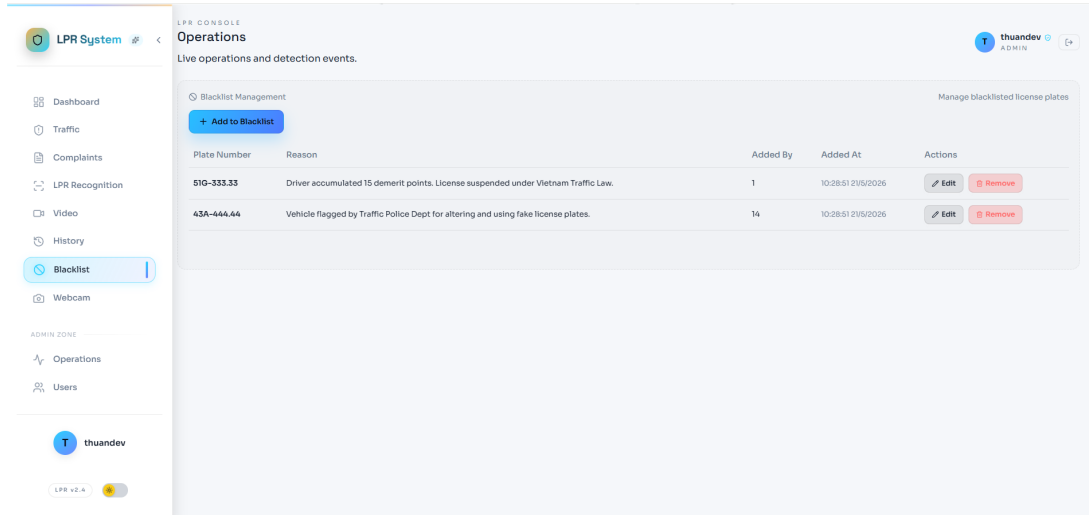
Đây là giao diện người dùng có thể xem lại lịch sử ảnh mình upload và được hệ thống xử lý thành công. Ngoài ra, giao diện dashboard cũng dùng dữ liệu trong mục history. Người dùng có thể tìm kiếm lịch sử lọc theo thời gian, xuất sang file CSV hoặc có thể xóa lịch sử tìm kiếm nếu muốn.



Hình 10: Giao diện lịch sử

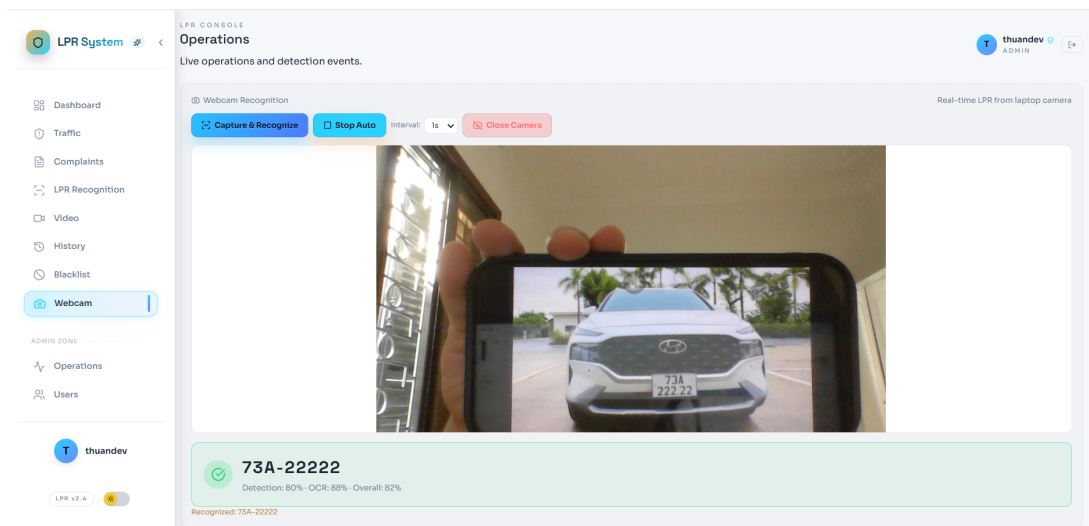
9 Giao diện Blacklist

Người dùng chỉ được phép xem các danh sách đang bị blacklist, riêng Admin là được quyền chỉnh sửa danh sách này như chỉnh sửa lỗi vi phạm và xóa người dùng. Blacklist này là phần phụ của việc xử lý lỗi nếu người dùng vi phạm vượt quá 12 điểm thì hệ thống sẽ tự động đưa vào.



Hình 11: Giao diện Blacklist

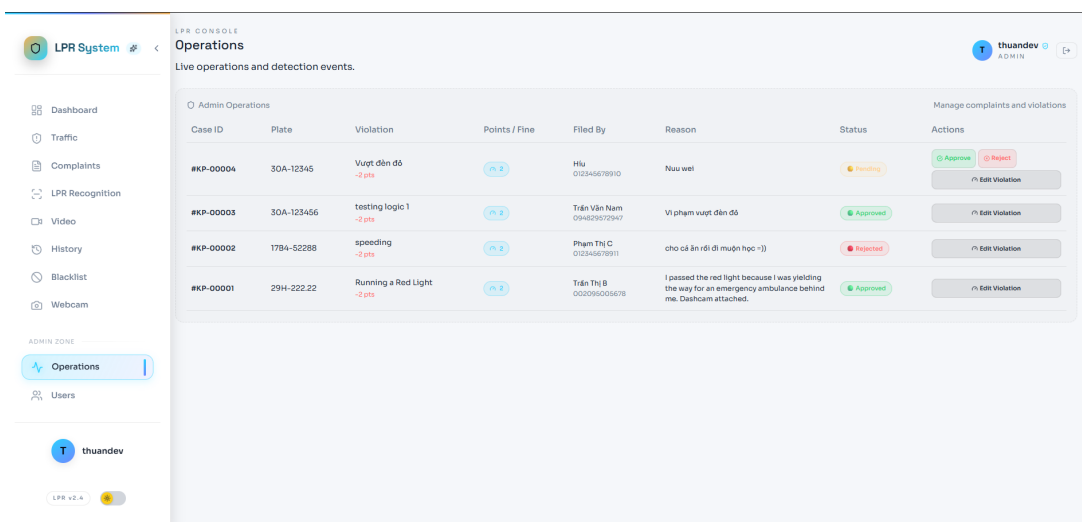
10 Giao diện webcam



Hình 12: Giao diện Webcam

11 Giao diện xử lý khiếu nại/nộp phạt

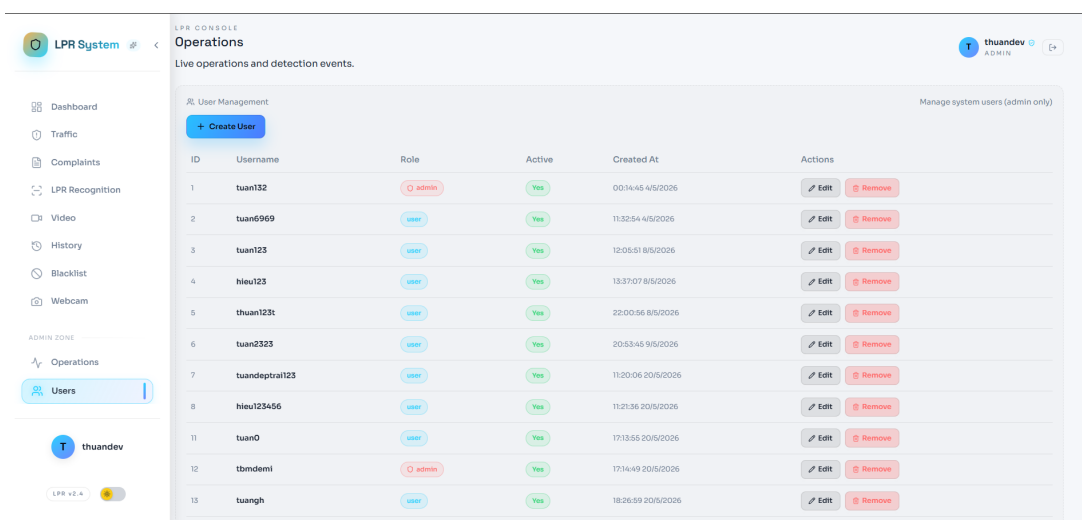
Khi người dùng tạo đơn khiếu nại/nộp phạt, Admin sẽ phải xử lý từng đơn như chấp nhận đơn hoặc từ chối. Ngoài ra có thể tùy chỉnh mức phạt thay đổi số điểm, thay đổi số tiền cần nộp phạt nếu có..



Hình 13: Giao diện Xử lý khiếu nại/ nộp phạt của Admin

12 Giao diện quản lý danh sách người dùng

Admin được phép tạo thêm người dùng mới, cấp quyền cho người dùng trở thành Admin hoặc User. Bên cạnh đó còn thêm tính năng xóa người dùng nếu người dùng đó không hoạt động, và thay đổi mật khẩu của người dùng...



Hình 14: Giao diện quản lý người dùng

IX MONITORING VÀ VẬN HÀNH

1 Mục tiêu monitoring

Hệ thống nhận diện biển số xe cần được giám sát liên tục vì có nhiều thành phần chạy đồng thời: frontend React, backend FastAPI, Redis, Celery Worker, MinIO, Prometheus, Grafana và Nginx reverse proxy. Việc monitoring giúp phát hiện sớm lỗi API, tình trạng quá tải tài nguyên, lỗi xử lý video và sự cố mất kết nối backend.

Các mục tiêu chính gồm:

- Theo dõi trạng thái hoạt động của backend FastAPI.
- Đo số lượng request, mã trạng thái HTTP và tốc độ phản hồi API.
- Theo dõi lỗi server 5xx để phát hiện lỗi nghiệp vụ hoặc lỗi hệ thống.
- Giám sát CPU, RAM và số lượng kết nối WebSocket realtime.
- Hỗ trợ vận hành qua logs, dashboard Grafana và cảnh báo Prometheus.

2 Kiến trúc monitoring

Backend FastAPI expose endpoint metrics tại:

`/metrics`

Prometheus định kỳ scrape metrics từ service backend trong Docker Compose:

```
job_name: lpr-backend
metrics_path: /metrics
targets:
  - backend:8000
```

Grafana được cấu hình sẵn datasource Prometheus tại:

`http://prometheus:9090`

Dashboard Grafana được provision tự động từ file:

`deploy/grafana/dashboards/lpr_dashboard.json`

Luồng monitoring tổng quát:

```
FastAPI Backend
  |
  | expose /metrics
  v
Prometheus
  |
  | query PromQL
  v
Grafana Dashboard
  |
  v
```

Theo dõi request, latency, error, CPU, RAM, WebSocket, alert

3 Các metrics chính

Bảng 35: Các metrics Prometheus của hệ thống

Metric	Ý nghĩa
http_requests_total	Tổng số HTTP request theo method, endpoint và mã trạng thái.
http_request_duration_seconds	Thời gian xử lý request, dùng để tính latency p50, p90, p99.
http_errors_total	Tổng số lỗi HTTP 5xx của backend.
active_websocket_connections	Số lượng kết nối WebSocket realtime đang hoạt động.
system_cpu_usage_percent	Phần trăm CPU đang sử dụng.
system_memory_usage_bytes	RAM used, total và available của hệ thống.
up	Trạng thái Prometheus scrape được backend hay không.

4 Dashboard Grafana

Dashboard Grafana của dự án hiển thị các nhóm thông tin quan trọng sau:

- **HTTP Request Rate:** số request mỗi giây, phân loại theo mã trạng thái 2xx, 4xx, 5xx.
- **API Response Time:** độ trễ API theo các percentile p50, p90 và p99.
- **Global Error Rate:** tỷ lệ lỗi 5xx trên tổng số request.
- **Error Count:** tốc độ phát sinh lỗi backend.
- **Active WebSocket Connections:** số kết nối realtime hiện tại.
- **CPU Usage:** mức sử dụng CPU.
- **Memory Usage:** RAM used, total và available.
- **Fire Alerts:** các cảnh báo Prometheus đang firing.

5 Cảnh báo Prometheus

Bảng 36: Các alert rule trong hệ thống

Alert	Điều kiện	Mức độ
HighErrorRate	Tỷ lệ lỗi 5xx lớn hơn 5% trong 2 phút	Warning
HighMemoryUsage	RAM sử dụng lớn hơn 85% trong 3 phút	Warning
BackendUnavailable	Prometheus không scrape được backend trong 1 phút	Critical
HighCpuUsage	CPU lớn hơn 80% trong 5 phút	Warning

6 Logging

Backend sử dụng structured JSON logging. Mỗi request được ghi log với các trường chính:

- **timestamp**: thời điểm phát sinh log.
- **level**: cấp độ log như INFO, WARNING, ERROR.
- **logger**: module phát sinh log.
- **message**: nội dung log.
- **endpoint**: API endpoint được gọi.
- **method**: HTTP method.
- **status_code**: mã phản hồi HTTP.
- **client_ip**: địa chỉ IP client.

Cách xem logs khi vận hành:

```
docker compose -f docker-compose.prod.yml logs -f backend
docker compose -f docker-compose.prod.yml logs -f worker
docker compose -f docker-compose.prod.yml logs -f nginx
```

7 Quy trình vận hành

Các bước kiểm tra hệ thống sau khi triển khai:

1. Kiểm tra trạng thái container:

```
docker compose -f docker-compose.prod.yml ps
```

2. Kiểm tra website qua Nginx:

```
http://localhost:8080
```

3. Kiểm tra Prometheus:

```
http://localhost:9090
```

4. Kiểm tra Grafana:

```
http://localhost:3000
```

5. Theo dõi logs backend và worker nếu có lỗi nhận diện hoặc xử lý video.

Dữ liệu runtime được lưu trong các Docker volume như Redis data, MinIO data, Prometheus data và Grafana data. Điều này giúp hạn chế mất dữ liệu khi container được restart.

X TRIỂN KHAI DOCKER CI & CD

1 Mục tiêu triển khai

Dự án được container hóa bằng Docker để đảm bảo môi trường chạy ổn định giữa máy phát triển, CI và production. Hệ thống CI/CD sử dụng GitHub Actions để tự động kiểm thử backend, build frontend, build Docker image và push image lên GitHub Container Registry.

2 Docker hóa backend

Backend sử dụng image nền:

```
python:3.10-slim
```

Dockerfile backend cài các thư viện hệ thống cần thiết cho xử lý ảnh, video và AI:

- `ffmpeg`: xử lý video.
- `libgl1, libglib2.0-0`: hỗ trợ OpenCV.
- `libgomp1`: hỗ trợ runtime cho các thư viện tính toán.
- `torch` và `torchvision` bản CPU.

Backend chạy bằng Uvicorn tại port 8000:

```
uvicorn src.backend.main:app --host 0.0.0.0 --port 8000
```

3 Docker hóa frontend

Frontend sử dụng multi-stage build:

- Stage 1 dùng `node:20-alpine` để cài dependency và build Vite.
- Stage 2 dùng `nginx:1.27-alpine` để serve static files.

Biến build quan trọng:

```
VITE_API_BASE
```

Biến này xác định base URL để frontend gọi API backend.

4 Docker Compose production

File triển khai production chính là:

`docker-compose.prod.yml`

Các service chính:

Bảng 37: Các service trong Docker Compose

Service	Vai trò
redis	Broker cho Celery và pub/sub cho WebSocket realtime.
minio	Lưu trữ ảnh, video upload và video đã xử lý.
backend	FastAPI API server, xử lý nghiệp vụ và expose metrics.
worker	Celery worker xử lý video bất đồng bộ.
frontend	React/Vite app được serve bằng Nginx.
prometheus	Thu thập và lưu trữ metrics.
grafana	Hiển thị dashboard monitoring.
nginx	Reverse proxy cho frontend, API backend và MinIO files.

Chạy toàn bộ hệ thống:

```
docker compose -f docker-compose.prod.yml up --build -d
```

Dừng hệ thống:

```
docker compose -f docker-compose.prod.yml down
```

5 Luồng CI/CD

Workflow CI/CD nằm tại:

`.github/workflows/ci-cd.yml`

Pipeline được kích hoạt khi:

- Có pull request.
- Có push vào branch `main`.

Pipeline gồm 3 job chính:

1. **Backend tests:** cài Python 3.10, cài thư viện hệ thống, cài dependency backend và chạy `pytest`.
2. **Frontend build:** cài Node.js 20, chạy `npm ci` và `npm run build`.
3. **Docker build push:** chỉ chạy khi push vào `main`; build và push Docker image backend/frontend lên GHCR.

Lệnh test backend trong CI:

```
pytest src/backend/tests -q --ignore=src/backend/tests/test_video_ocr.py
```

Lệnh build frontend trong CI:

```
npm ci
npm run build
```

6 Docker image registry

Image được push lên GitHub Container Registry với 2 loại tag:

- `latest`: phiên bản mới nhất trên branch main.
- `$github.sha`: tag theo commit SHA, dùng để rollback chính xác.

Cấu trúc image:

```
ghcr.io/<owner>/<repo>/backend:<tag>
ghcr.io/<owner>/<repo>/frontend:<tag>
```

7 Quy trình CD trên server

Trên server production, cập nhật biến image trong file môi trường:

```
BACKEND_IMAGE=ghcr.io/Tuan-No-UET-2306/btl/backend:latest
FRONTEND_IMAGE=ghcr.io/Tuan-No-UET-2306/btl/frontend:latest
```

Sau đó pull image và restart service:

```
docker login ghcr.io
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

Kiểm tra trạng thái sau deploy:

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f backend
```

8 Rollback

Do mỗi image có tag theo commit SHA, rollback có thể thực hiện bằng cách đổi tag image về commit ổn định trước đó:

```
BACKEND_IMAGE=ghcr.io/Tuan-No-UET-2306/btl/backend:latest
FRONTEND_IMAGE=ghcr.io/Tuan-No-UET-2306/btl/frontend:latest
```

```
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

9 Bảo mật khi triển khai

Một số lưu ý quan trọng:

- Không commit file `.env` hoặc secret lên GitHub.
- Đổi `JWT_SECRET_KEY`, tài khoản admin và mật khẩu Grafana khi deploy production.
- Chỉ cấu hình `CORS_ORIGINS` cho domain thật.
- Không public Prometheus, Grafana và MinIO Console nếu chưa có xác thực.
- Sử dụng HTTPS cho domain production.
- Backup định kỳ database và dữ liệu MinIO.

XI KIỂM THỬ VÀ ĐÁNH GIÁ

1 Mục tiêu kiểm thử

Mục tiêu kiểm thử là đảm bảo hệ thống nhận diện biển số xe hoạt động đúng theo yêu cầu, bao gồm các chức năng đăng nhập, phân quyền, nhận diện biển số từ ảnh, webcam, video, lưu lịch sử, quản lý blacklist, tra cứu giao thông và vận hành bằng Docker.

2 Môi trường kiểm thử

Bảng 38: Môi trường kiểm thử

Thành phần	Cấu hình / Công nghệ
Backend	FastAPI, Python 3.10, Uvicorn
Frontend	React, Vite, Tailwind CSS
Database	PostgreSQL / Supabase
Storage	MinIO
Background Job	Celery, Redis
AI Model	YOLOv5, OCR, ONNXRuntime, OpenCV
Triển khai	Docker, Docker Compose, Nginx
Monitoring	Prometheus, Grafana

3 Các loại kiểm thử

- **Kiểm thử đơn vị:** kiểm tra các service, repository và API backend.
- **Kiểm thử tích hợp:** kiểm tra luồng kết nối giữa frontend, backend, database, Redis và MinIO.
- **Kiểm thử chức năng:** kiểm tra các chức năng nghiệp vụ chính của hệ thống.
- **Kiểm thử triển khai:** kiểm tra khả năng build, chạy và restart hệ thống bằng Docker Compose.
- **Kiểm thử giao diện:** kiểm tra khả năng sử dụng trên các màn hình chính.

4 Kiểm thử tự động

Backend được kiểm thử bằng `pytest`. Lệnh chạy test:

```
pytest src/backend/tests -q --ignore=src/backend/tests/test_video_ocr.py
```

Frontend được kiểm tra bằng quá trình build production:

```
cd src/frontend
npm ci
npm run build
```

Pipeline CI/CD cũng tự động chạy các bước kiểm thử này khi có pull request hoặc push lên branch `main`.

5 Kiểm thử chức năng

Bảng 39: Kịch bản kiểm thử chức năng

Chức năng	Nội dung kiểm thử	Kết quả
Đăng ký, đăng nhập	Tạo tài khoản, đăng nhập, kiểm tra JWT token	Đạt
Phân quyền	User thường không truy cập được chức năng admin	Đạt
Nhận diện ảnh	Upload ảnh xe, trả về biển số và bounding box	Đạt
Nhận diện webcam	Gửi frame realtime và hiển thị kết quả nhận diện	Đạt
Nhận diện video	Upload video, queue xử lý, trả về video đã xử lý	Đạt
Lịch sử nhận diện	Lưu, tìm kiếm, lọc và export CSV	Đạt
Blacklist	Thêm, xoá, kiểm tra biển số trong danh sách đen	Đạt
Tra cứu giao thông	Tra cứu phương tiện, vi phạm và điểm trừ	Đạt
Monitoring	Prometheus scrape metrics, Grafana hiển thị dashboard	Đạt
Docker Compose	Build và chạy toàn bộ service bằng một lệnh	Đạt

6 Đánh giá kết quả

Hệ thống đáp ứng được các yêu cầu chính của bài toán nhận diện biển số xe. Các chức năng nghiệp vụ như đăng nhập, phân quyền, nhận diện ảnh, nhận diện video, quản lý blacklist và tra cứu giao thông hoạt động ổn định trong môi trường kiểm thử.

Tuy nhiên, kết quả nhận diện biển số vẫn phụ thuộc vào chất lượng ảnh đầu vào. Các trường hợp ảnh bị mờ, thiếu sáng, biển số bị che khuất hoặc góc chụp quá nghiêng có thể làm giảm độ chính xác của mô hình.

7 Hạn chế

- Chưa tối ưu hoàn toàn cho tải lớn nhiều người dùng đồng thời.
- Hiệu quả nhận diện phụ thuộc vào chất lượng camera và điều kiện ánh sáng.
- Việc xử lý video có thể tốn CPU/RAM nếu video dài hoặc độ phân giải cao.
- Chưa triển khai đầy đủ Alertmanager để gửi cảnh báo qua email hoặc Telegram.

XII BẢO MẬT VÀ RỦI RO

1 Các cơ chế bảo mật đã áp dụng

- **Xác thực JWT:** người dùng đăng nhập thành công sẽ nhận token để gọi các API yêu cầu xác thực.
- **Phân quyền user/admin:** các chức năng quản trị người dùng chỉ dành cho tài khoản admin.
- **Mã hoá mật khẩu:** mật khẩu không lưu trực tiếp mà được hash trước khi lưu vào database.
- **CORS:** chỉ cho phép các domain frontend hợp lệ gọi API backend.
- **Biến môi trường:** thông tin nhạy cảm như database URL, JWT secret, MinIO key được cấu hình qua file `.env`.
- **Reverse proxy Nginx:** backend và MinIO được truy cập thông qua proxy, giúp kiểm soát đường dẫn public.
- **Docker isolation:** các service chạy tách biệt trong container.
- **Structured logging:** hỗ trợ truy vết lỗi và kiểm tra hoạt động hệ thống.

2 Rủi ro và biện pháp giảm thiểu

Bảng 40: Rủi ro bảo mật và vận hành

Rủi ro	Ảnh hưởng	Biện pháp giảm thiểu
Lộ file .env	Mất thông tin database, JWT secret, MinIO key	Không commit .env, dùng GitHub Secrets khi deploy
JWT secret yếu	Token có thể bị giả mạo	Sử dụng secret dài, ngẫu nhiên và thay đổi định kỳ
Upload file độc hại	Gây lỗi xử lý hoặc chiếm tài nguyên server	Giới hạn loại file, dung lượng file và kiểm tra MIME type
Quá tải khi xử lý video	Backend/worker bị chậm hoặc crash	Dùng Celery worker, giới hạn số frame xử lý và giới hạn concurrency
Sai kết quả nhận diện	Ảnh hưởng nghiệp vụ tra cứu/vi phạm	Lưu lịch sử, cho phép kiểm tra lại và cải thiện dữ liệu huấn luyện
Public Grafana/Prometheus	Lộ thông tin vận hành hệ thống	Không public dashboard hoặc đặt xác thực mạnh
Redis hoặc MinIO lỗi	Video không xử lý được hoặc không lưu được file	Healthcheck, restart policy và backup dữ liệu định kỳ
CORS cấu hình sai	Website không gọi được API hoặc bị gọi từ domain lạ	Chỉ cấu hình domain production hợp lệ

3 Khuyến nghị bảo mật khi triển khai production

- Bắt buộc sử dụng HTTPS cho domain production.
- Đổi mật khẩu admin mặc định trước khi public hệ thống.
- Không public MinIO Console, Prometheus và Grafana nếu chưa có xác thực.
- Backup database và object storage định kỳ.
- Giới hạn dung lượng upload video để tránh tấn công làm đầy bộ nhớ.
- Theo dõi logs và dashboard Grafana để phát hiện bất thường.

XIII PHÂN CÔNG NHÓM

1 Phân công nhóm

Nhóm gồm 4 thành viên. Công việc được phân chia theo các mảng chính: backend, frontend, AI xử lý ảnh/video, triển khai và tài liệu.

Bảng 41: Bảng phân công công việc nhóm

Thành viên	Nhiệm vụ chính	Kết quả phụ trách
Trần Công Tuấn Phạm Văn Vương Thuận	Phân tích yêu cầu, thiết kế kiến trúc, xây dựng backend FastAPI	API xác thực, phân quyền, lịch sử nhận diện, database schema
Nguyễn Đắc Trung Hiếu Phạm Văn Vương Thuận	Xây dựng giao diện frontend React	Màn hình đăng nhập, dashboard, nhận diện ảnh/video, lịch sử, blacklist
Trịnh Bình Minh Trần Công Tuấn Phạm Văn Vương Thuận	Tích hợp mô hình AI, xử lý ảnh, webcam và video	YOLOv5/OCR, xử lý frame, nhận diện biển số, Celery video task
Trần Công Tuấn Nguyễn Đắc Trung Hiếu	Triển khai hệ thống, CI/CD, monitoring và tài liệu báo cáo	Docker Compose, Nginx, GitHub Actions, Prometheus, Grafana, slide bảo vệ

2 Kế hoạch thực hiện

Bảng 42: Kế hoạch thực hiện dự án

Giai đoạn	Công việc	Kết quả
Giai đoạn 1	Khảo sát bài toán, xác định chức năng, thiết kế kiến trúc	Đặc tả yêu cầu, sơ đồ hệ thống
Giai đoạn 2	Xây dựng backend, database, xác thực và phân quyền	API nền tảng hoạt động
Giai đoạn 3	Xây dựng frontend và kết nối API	Giao diện người dùng hoàn chỉnh
Giai đoạn 4	Tích hợp nhận diện biển số từ ảnh, webcam và video	Chức năng AI hoạt động
Giai đoạn 5	Docker hóa, CI/CD, monitoring và kiểm thử	Hệ thống chạy ổn định bằng Docker
Giai đoạn 6	Hoàn thiện báo cáo, slide và luyện tập bảo vệ	Sẵn sàng trình bày trước hội đồng

3 Kịch bản demo

1. Đăng nhập vào hệ thống bằng tài khoản người dùng.
2. Upload ảnh xe và hiển thị kết quả nhận diện biển số.
3. Chạy nhận diện realtime bằng webcam.
4. Upload video, queue xử lý và xem kết quả video đã nhận diện.
5. Kiểm tra lịch sử nhận diện và export dữ liệu.
6. Thêm biển số vào blacklist và kiểm tra cảnh báo.
7. Đăng nhập tài khoản admin để quản lý người dùng.
8. Mở Grafana để minh họa dashboard monitoring.